

4

Computational Thinking and Getting Started with Python

4.1 INTRODUCTION TO PROBLEM-SOLVING

Computers help us solve problems but they have no intelligence or thinking power. A computer performs many tasks exactly in the manner as instructed. However, before a problem can be tackled, we need to understand the problem and the ways to solve it. So, in order to instruct a computer correctly, the user must have clear understanding of the problem to be solved.

Irrespective of the programming language being used, the first and foremost thing while writing a program is to analyze the problem well and try to solve it in a logical manner. The process of writing a program is called "Programming". Thus, when we write a program, we need to first clearly understand the problem as well as its solution, otherwise we will end up writing incorrect or inefficient codes. When we write a program, we are writing a set of instructions to solve a problem involving data processing. These instructions must be executed by the computer to solve the problem. If the instructions are correct and given in right sequence, then the problem will be solved properly by the computer. But if the instructions are incorrect or in the wrong sequence, then the problem will not be solved correctly or will not be solved at all.

Therefore, writing correct instructions in proper sequence is extremely important. It means that before we start writing a program, the steps for solving the problem should be clear in our mind. Precise step-by-step instructions should be inputted to the computer system to solve the problem. Thus, proper and flawless execution of any task inside a computer depends on how correctly and precisely we define the problem, design a solution (algorithm) and implement the solution (program) using a programming language. Problem-solving, therefore, is the process of identifying a problem, developing an algorithm for the identified problem and finally implementing the algorithm to develop a computer program.



Fig. 4.1: Computer Programming

4.2 STEPS FOR PROBLEM-SOLVING

There are a number of problems in our day-to-day life, such as developing marksheet or calculating simple interest, and all problems require proper problem-solving mechanism. Problem-solving is the act of defining a problem, determining the cause of the problem, identifying, prioritizing and selecting alternatives for a solution, and implementing a solution.

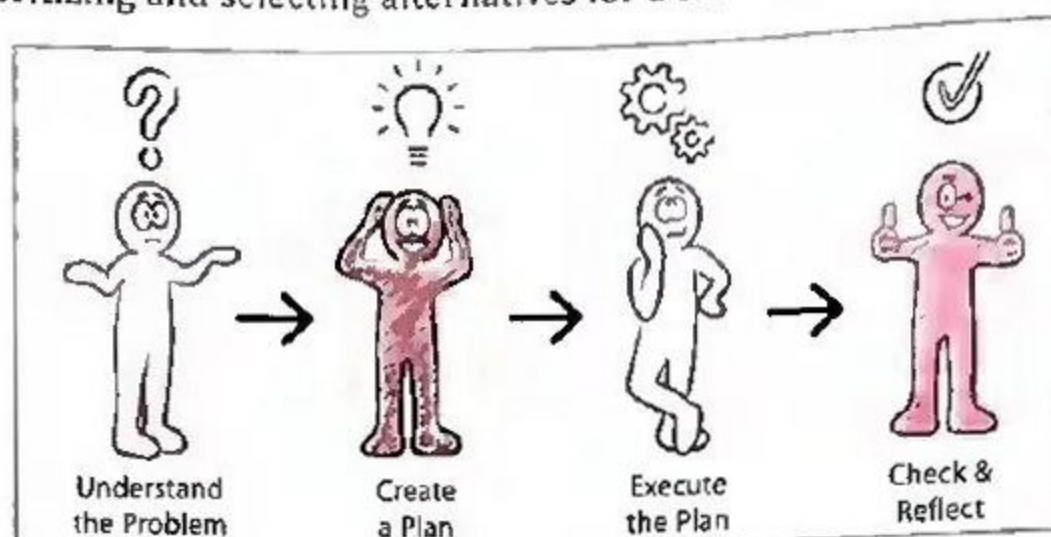


Fig. 4.2: Concept of Problem-solving

When problems are straightforward and easy, we can find solutions without difficulty. But a complex problem requires a methodical approach to find the right solution. In other words, we have to apply problem-solving techniques. Problem-solving begins with the precise identification of the problem and ends with a complete working solution in terms of a program or software. The key steps required for solving a problem using a computer are shown in Fig. 4.3 and discussed in the following sub-sections.

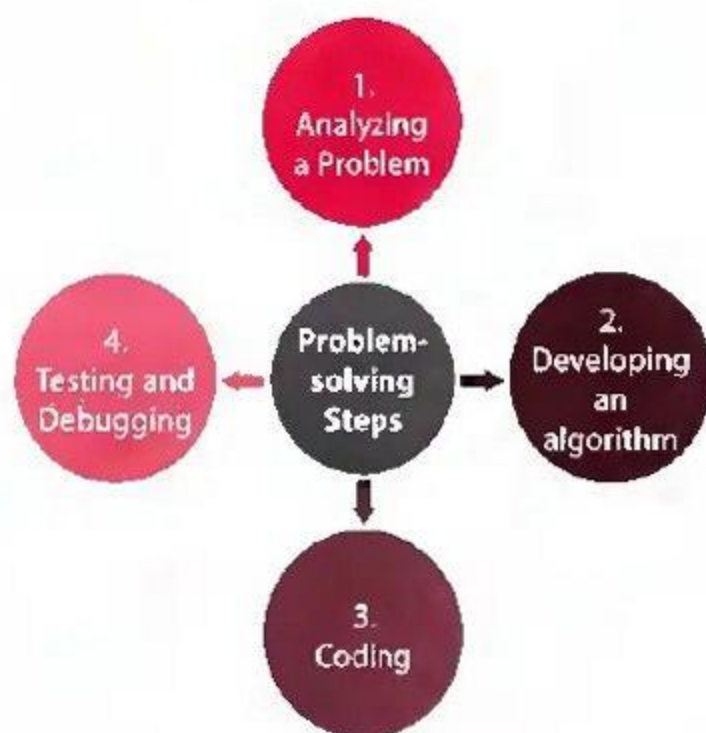


Fig. 4.3: Steps for Problem-solving

4.2.1 Analyzing a Problem

It is important to clearly understand a problem before we begin to find a solution to it. If we are not clear as to what is to be solved, we may end up developing a program which may not solve our purpose. Thus, we need to read and analyze the problem statement carefully in order to list the principal components of the problem and decide the core functionalities that our solution should have. By analyzing a problem, we would be able to figure out what are the inputs that our program should accept and the outputs that it should produce.

4.2.2 Developing an Algorithm

It is essential to devise a solution before writing a program code for a given problem. The set of sequential steps usually represented in natural language is called an algorithm. We can imagine an algorithm like a very well-written recipe for a dish with clearly defined steps which, if followed, one will end up preparing a good dish. We start with a tentative solution plan and keep on refining the algorithm until the algorithm is able to capture all aspects of the desired solution. For a given problem, more than one algorithm is possible and we have to select the most suitable solution. The algorithm is discussed in the next section.

4.2.3 Coding

After finalizing the algorithm, we need to convert the algorithm into a format which can be understood by the computer to generate the desired solution. Different high-level programming languages can be used for writing a program. It is equally important to record details of the coding procedures followed and document the solution. This is helpful when revisiting the programs at a later stage.

4.2.4 Testing and Debugging

The program created should be tested on various parameters. The program should meet the requirements of the user. It must respond within the expected time. It should generate correct output for all possible inputs. In the presence of syntactical errors, no output will be obtained. In case the output generated is incorrect, then the program should be checked for logical errors, if any. The software industry follows standardized testing methods like unit or component testing, integration testing, system testing, and acceptance testing while developing complex applications. This is to ensure that the software meets all the business and technical requirements and works as expected. The errors or defects found in the testing phases are debugged or rectified and the program is again tested. This continues till all the errors are removed from the program. Once the software application has been developed, tested and delivered to the user, still problems in terms of functioning can crop up and need to be resolved from time to time. The maintenance of the solution, thus, involves fixing the problems faced by the user, answering the queries of the user and even serving the request for addition or modification of features.

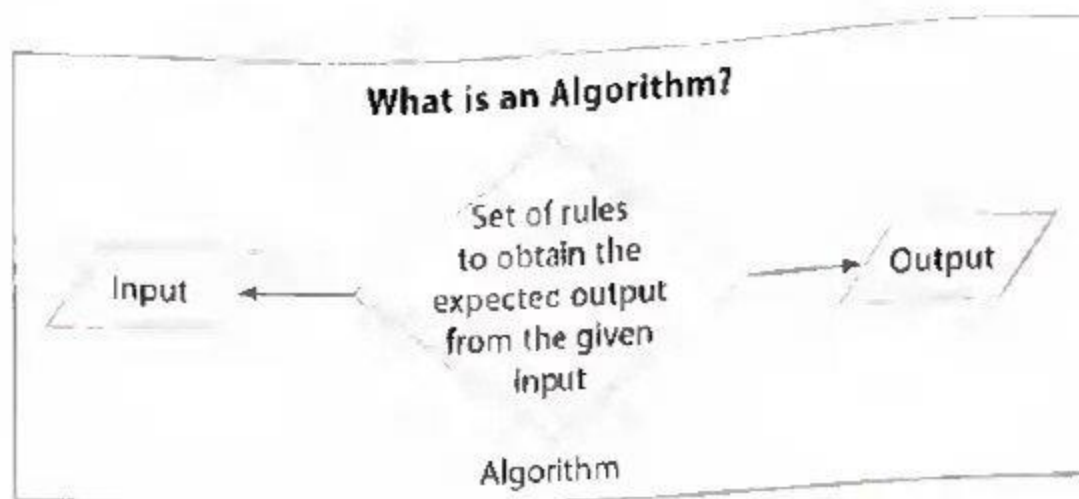
The problem-solving stages explained above help in developing computational thinking and, in turn, offer proper, accurate and complete solution to a given problem.

4.3 ALGORITHM

In our day-to-day life, we perform activities by following a certain sequence of steps. Examples of activities include getting ready for school, making breakfast, riding a bicycle, wearing a tie, solving a puzzle and so on. To complete each activity, we follow a sequence of steps.

Suppose the following steps are required for the activity 'Riding a car':

1. Move the car out of the garage
2. Sit in the driver's seat
3. Put key in the ignition and start the car
4. Press the clutch and put gear in the correct position
5. Accelerate and start driving
6. Use brakes whenever needed
7. Stop on reaching the destination.



Let us look at another example of finding the Greatest Common Divisor (GCD) of two numbers 45 and 54.

Note: GCD is the largest number that divides both the given numbers.

Step 1: Find the numbers (divisors) which can divide the given numbers:

Divisors of 45 are 1, 3, 5, 9, 15 and 45.

Divisors of 54 are 1, 2, 3, 6, 9, 18, 27 and 54.

Step 2: Find the largest common number from these two lists, which is 9.

Thus, GCD of 45 and 54 is 9.

Hence, it is clear that we need to follow a sequence of steps to accomplish the task. Such a finite sequence of steps required to get the desired output is called an algorithm. It will lead to desired result in a finite amount of time, if followed correctly. Algorithm has a definite beginning and a definite end, and consists of a finite number of steps.

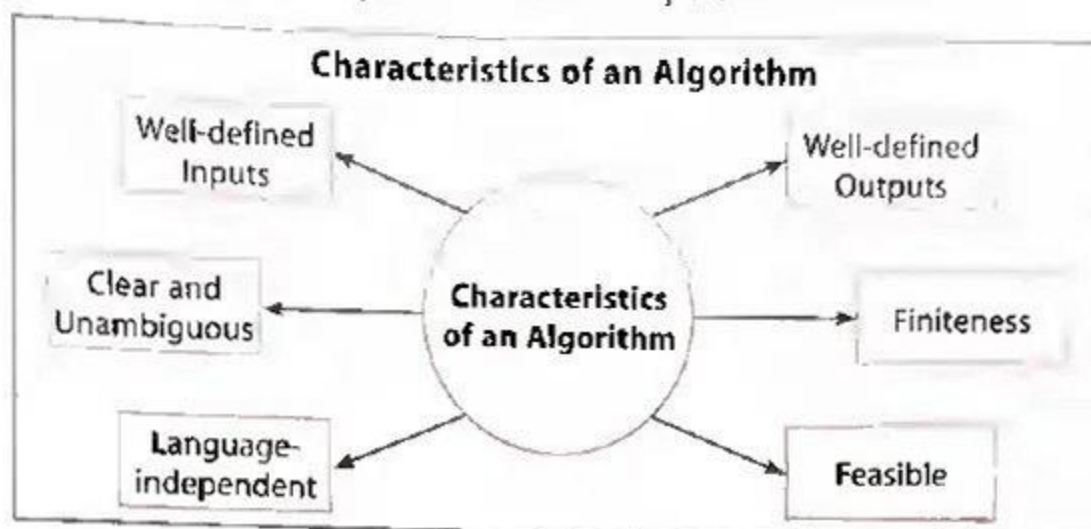
4.3.1 Need for an Algorithm

A programmer writes a program to instruct the computer to do certain tasks as desired. The computer then follows the steps written in the program code. Thus, the programmer first prepares a road map of the program to be written before actually writing the code. Without a road map, the programmer may not be able to clearly visualize the instructions to be written and may end up developing a program which may not work as expected. Such a road map is nothing but the algorithm which is the building block of a computer program. For example, searching using a search engine, sending a message, finding a word in a document, booking a taxi through an app, using online banking, playing computer games—all are based on algorithms.

Writing an algorithm is mostly considered as a first step to programming. Once we have an algorithm to solve a problem, we can write the computer program for giving instructions to the computer in high-level language. If the algorithm is correct, computer will run the program correctly, every time. So, the purpose of using an algorithm is to increase reliability, accuracy and efficiency of obtaining solutions. Thus, algorithmic problem-solving actually comes in two phases: derivation of an algorithm that solves the problem and conversion of the algorithm into code. The latter, usually known as coding, is comparatively easier since the logic is already present—it is just a matter of ensuring that the syntax rules of the programming language are adhered to.

(A) Characteristics of a good algorithm

- Precision — the steps are precisely stated or defined.
- Uniqueness — results of each step are well defined and only depend on the input and the result of the preceding steps.
- Finiteness — the algorithm always stops after a finite number of steps.
- Input — the algorithm receives some input.
- Output — the algorithm produces some output.



(B) While writing an algorithm, it is required to clearly identify the following:

- The input to be taken from the user.
- Processing or computation to be performed to get the desired result.
- The output desired by the user.

4.4 FLOW CHARTS—VISUAL REPRESENTATION OF AN ALGORITHM

A flow chart is the pictorial representation of a task. It is defined as a visual representation of the sequence of steps and decisions needed to perform a process/task. A flow chart is used to represent how a program works. In other words, a flow chart depicts the “flow” of a program, i.e., the breakdown of a task into separate steps as shown in Fig. 4.4. It shows all the steps of a process. Being visual, it helps in understanding the logic of a task quickly.

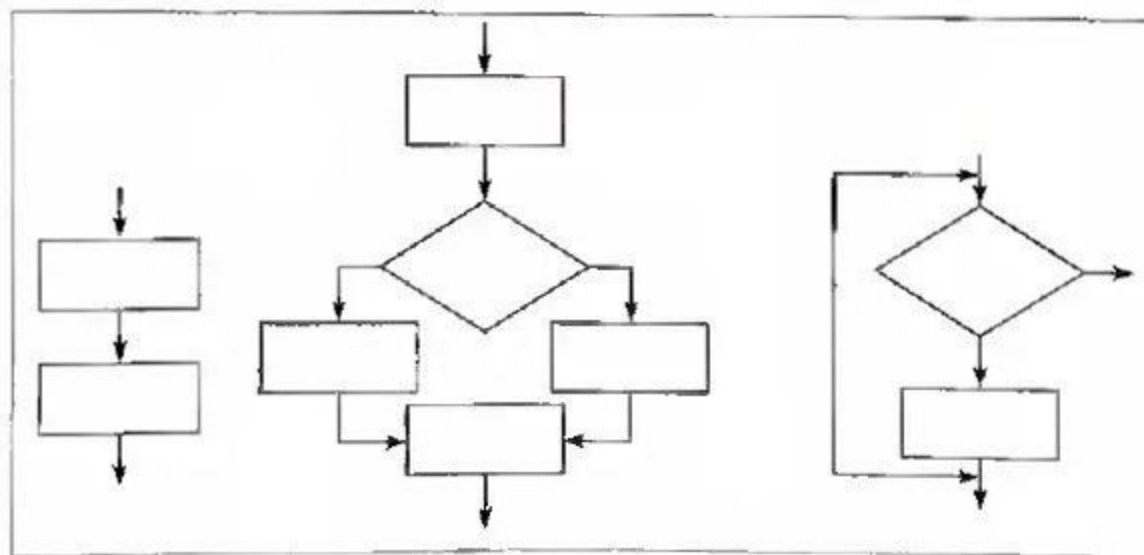


Fig. 4.4: A Flow Chart

Thus, flow charts are used in analyzing, designing, documenting or managing a process or program. Specific parts of the flow chart represent specific parts of your code. A flow chart, when translated into a proper computer language, results in a complete program.

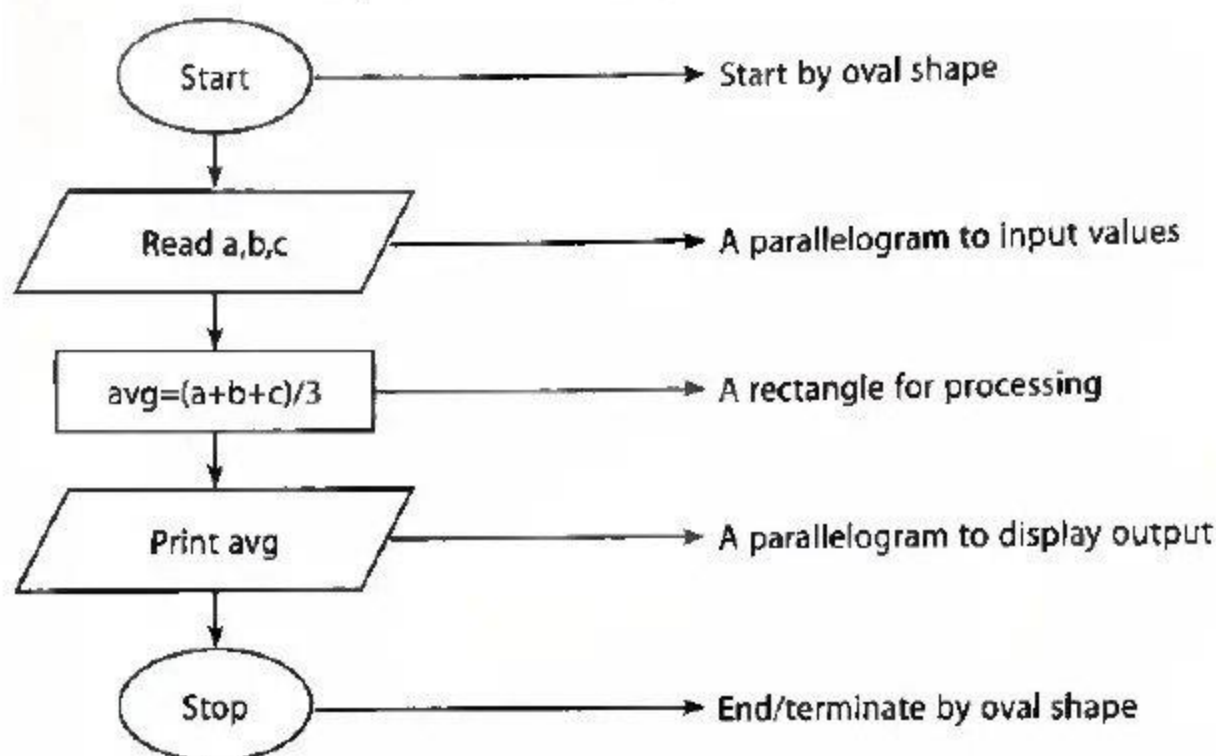
Table 4.1: Commonly-used Symbols in a Flow Chart

Symbol	Name	What it does in the code
	Start/End	Ovals show a start point or end point in the code
	Connection	Arrows show connection between different parts of the code
	Process	Rectangles show processes e.g., calculations (most things the computer does that do not involve an input, output or decision)
	Input/Output	Parallelograms show inputs and outputs (remember print is normally an input)
	Conditional/Decision	Diamonds show a decision/condition (this normally depicts if, if-else/if elif-else, while and for)

Let us understand flow chart through an example.

Example 1:

Draw a flow chart to find average of three numbers.

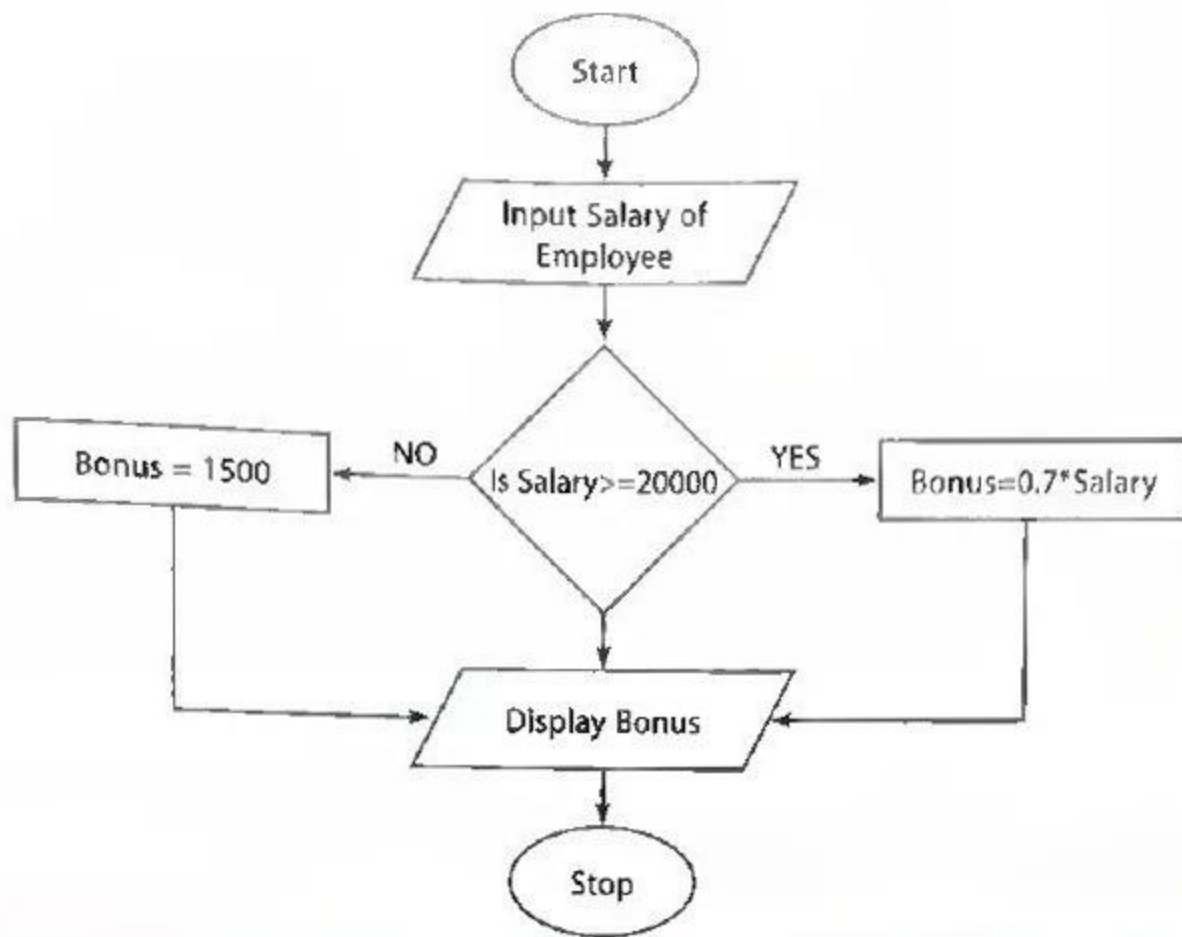


To solve this problem, we will ask the user to enter three values which get stored in three variables, a, b, c respectively. We will then calculate the average of three variables and store it in a variable 'avg'. Finally, we will print the value stored in the variable 'avg'.

Let us see another example to create a flow chart.

Example 2:

Speed Net Ltd. plans to give a 7% year-end bonus to each of its employees earning ₹ 20,000 or more per month and a fixed ₹ 1,500 bonus to the remaining employees. Draw a flow chart for calculating the bonus for an employee.



CTM: Flow chart is the pictorial representation of step-by-step solution to a given problem.

4.4.1 Pseudo-code

Pseudo-code is used to express our logic independent of any programming language. As the name suggests, it is not a real code; it is an informal approach in English-like language about how we will solve a problem.

There are no rules associated with writing a pseudo-code; it could be a list of instructions in English-like language or in any programming language. Using it, we can easily break down a bigger problem into smaller modules or sub-parts and give us better clarity about how we can solve a particular program or a process. This can be explained using a real-life example of making a phone call from a smartphone. Another example we have taken is sending an astronaut to the Moon for navigation as shown in Fig. 4.5.

Making a phone call



1. Unlock Phone
2. Open Contacts
3. Search for Contact
4. Place a Call
5. End Call

Pseudo-code

Sending Man to the Moon



1. Launch
2. Navigate to the Moon
3. Land on the Moon

Pseudo-code

Fig. 4.5: Examples of Pseudo-code

As is evident from the previous figure, the pseudo-code defined the steps involved in solving the above problem in simple steps using English-like structure. A good pseudo-code should:

- not be in a specific coding language,
- draft the structure of your code,
- be understandable to humans.

Example 3:

Write a pseudo-code that reads marks of ten students and displays average of marks.

Ans. Set total to zero

Set marks counter to one

While marks counter is less than or equal to ten

Input the next marks

Add the marks into the total

Set the average by dividing total by ten

Display the average.

Example 4:

Write a pseudo-code along with Python code to get a user's name as the input and then concatenate this name with "Hello" message as "Hello [Name]".

Ans.

Pseudo-code	Python code
input username	name = input("What is your name?")
output "Hello username"	print("Hello" + name)

CTM: Pseudo-code is a simple and informal way of writing programming code in English. There are no standard rules to write Pseudo-code.

4.5 WHAT IS COMPUTATIONAL THINKING

If we start writing a program without knowing how to solve the problem, we can never write the correct program. This leads us to the concept of Computational Thinking. Wikipedia defines Computational Thinking as the thought processes involved in formulating a problem and expressing its solution(s) in a way that a man or machine can effectively carry out. So, to write any program, first we have to think computationally—about the problem and its solution—and then we have to write the program in a programming language.

But every day, in all aspects of our lives, we perform Computational Thinking. Think of the following actions and decisions that go into them:

- You want to make yourself a cup of tea or coffee.
- You want to buy a car.
- You want to change job.
- You want to move to another city.
- You want to buy a house.
- You want to write a book.
- You want to create an app.

All the above situations require thought processing for finding the best solution and, thus, Computational Thinking becomes a crucial aspect.

Computational Thinking allows us to take a complex problem, understand what it is, and develop possible solutions. We can then present these solutions in a way that a computer, a human, or both, can understand.

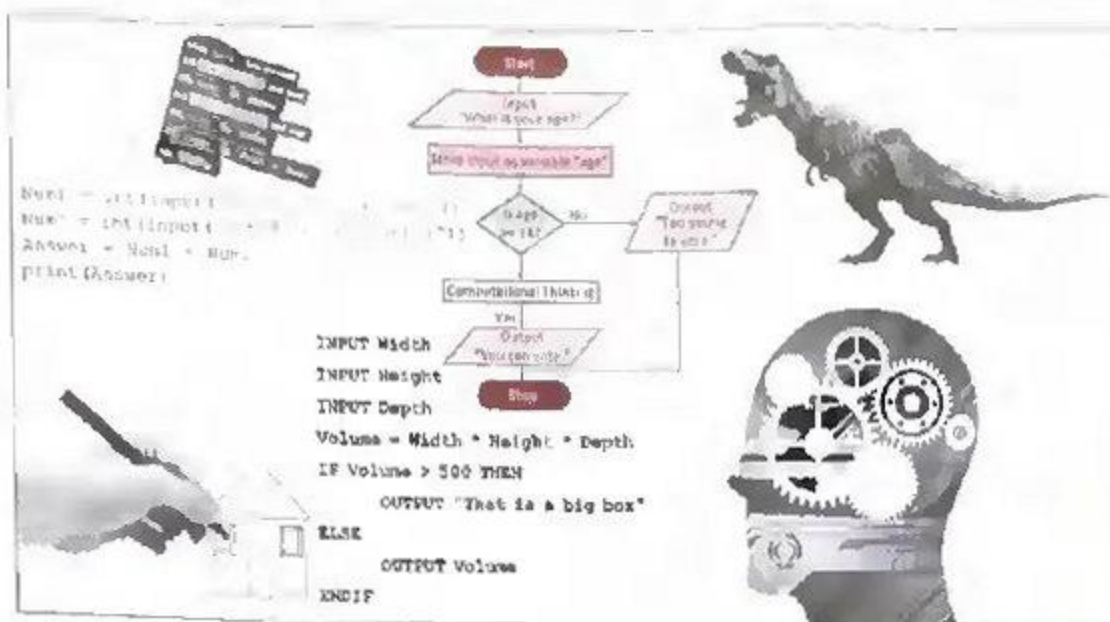


Fig. 4.6: Computational Thinking

The process of planning and reaching the goals involves Computational Thinking of some kind. Computational Thinking involves taking a complex problem and breaking it down into a series of small, more manageable problems (**decomposition**). Each of these smaller problems can then be looked at individually, considering how similar problems have been solved previously (**pattern recognition**) and focusing only on the important details, while ignoring irrelevant information (**abstraction**). Next, simple steps or rules to solve each of the smaller problems can be designed (**algorithms**).

Let us discuss the components of Computational Thinking in detail.

4.6 COMPONENTS OF COMPUTATIONAL THINKING

Computational Thinking is a technique used to solve problems logically and consists of four key components or pillars described as follows:

- ☞ Decomposition
- ☞ Pattern Recognition
- ☞ Abstraction
- ☞ Algorithms

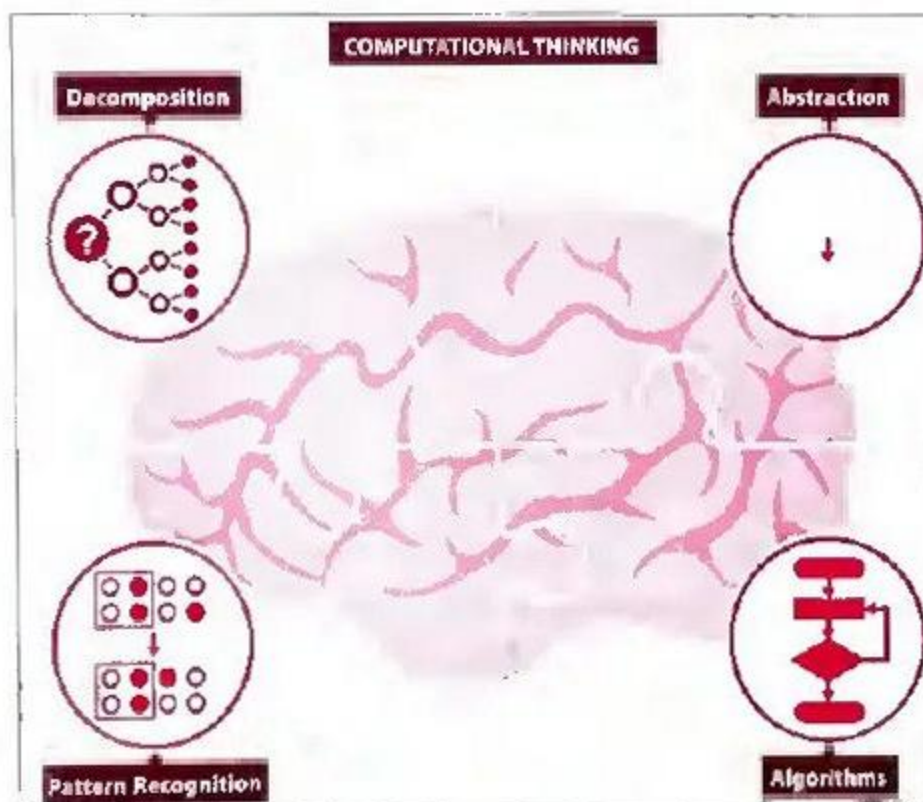


Fig. 4.7: Pillars of Computational Thinking

4.6.1 Decomposition

Decomposition is the process of breaking down a complex problem or system into smaller, easily manageable parts. These smaller problems are solved one after the other until the bigger complex problem is solved.

This stage involves breaking down the problem into smaller components so that they can be tackled easily. The more we can break down a problem, the easier it is to solve.

Some real-life examples of the decomposition process are:

- ☛ When we taste an unfamiliar dish and identify several ingredients based on the flavour, we are decomposing that dish into its individual ingredients.
- ☛ When we give someone directions to our house, we are decomposing the process of getting from one place to another (e.g., city, state, etc.).
- ☛ When we break a course project into several steps, we are decomposing the task into smaller, more manageable sub-tasks.
- ☛ In Mathematics, we can decompose a number such as 256.37 as follows:
 $2 \cdot 10^2 + 5 \cdot 10^1 + 6 \cdot 10^0 + 3 \cdot 10^{-1} + 7 \cdot 10^{-2}$

Let us look at a more specific example—the equation to work out the roots of a quadratic equation.

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

On first look, solving this equation might appear a little scary but if we decompose it, we will stand a better chance to solve it:

- | | |
|---|--|
| 1. b^2 | 2. $4ac$ |
| 3. $b^2 - 4ac$ | 4. $\sqrt{b^2 - 4ac}$ |
| 5. $-b \pm \sqrt{b^2 - 4ac}$ | 6. $2a$ |
| 7. $x_1 = \frac{-b + \sqrt{b^2 - 4ac}}{2a}$ | 8. repeat for $x_2 = \frac{-b - \sqrt{b^2 - 4ac}}{2a}$ |

By noting down the steps to solve a problem, we can often recognize patterns, and by giving a list of steps, we are one step closer to creating an algorithm.

This stage also allows us to develop a better understanding of the problem we face by identifying all the components in detail.

4.6.2 Pattern Recognition

Pattern Recognition is the next process in succession that helps in identifying patterns or trends within a problem. These patterns can help solve the larger problem more effectively. Once we have decomposed a complex problem into smaller problems, the next step is to look for distinguishing attributes (colour, shape, size), extract features or matching patterns. We look for patterns when we play games to decide when to do certain things. Based on experience, we develop shortcuts for mapping problem characteristics to arrive at a solution.

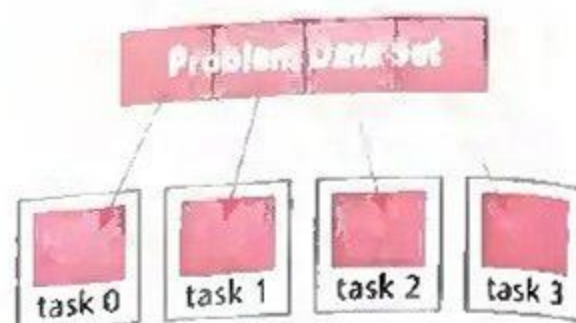


Fig. 4.8: Decomposition Process

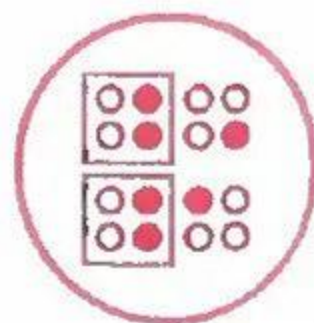


Fig. 4.9: Pattern Recognition

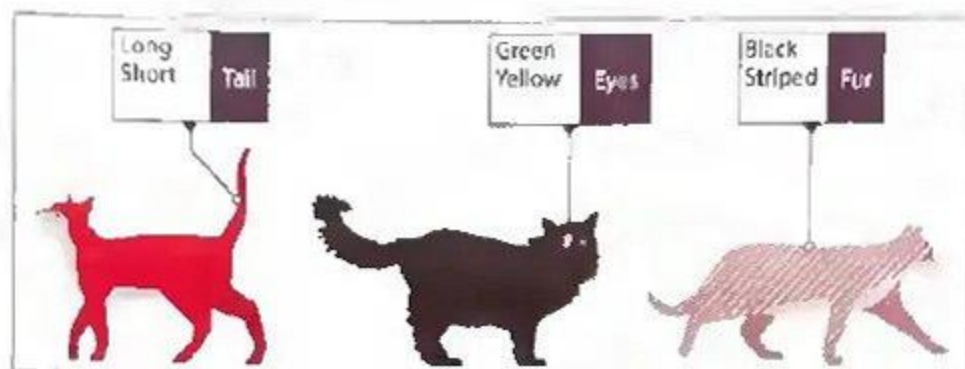
What are patterns

Imagine that we want to draw a series of cats.

All cats share common characteristics. Among other things, **they all have eyes, tails and fur**. They also like to eat fish and make meowing sounds. Because we know that all cats have eyes, tails and fur, we can make a good attempt at drawing a cat simply by including these common characteristics.

In Computational Thinking, these characteristics are known as patterns. **Once we know how to describe one cat, we can describe others by simply following this pattern.** The only things that are different are the specifics:

- one cat may have green eyes, a long tail and black fur
- another cat may have yellow eyes, a short tail and striped fur



If we want to draw a number of cats, finding a pattern to describe cats in general, *for example*, they all have eyes, tails and fur, makes this task quicker and easier.

We know that all cats follow this pattern, so we don't have to stop each time we start to draw a new cat to work this out. From the patterns we know cats follow, we can quickly draw several cats.



We experience pattern recognition in everyday situations like:

- ☞ Drivers look for patterns in traffic to decide whether and when to switch lanes.
- ☞ People look for patterns in stock prices to decide when to buy or sell.
- ☞ Scientists look for patterns in data to derive theories and models.
- ☞ We look for patterns and learn from them to avoid repeating the same mistake.

Patterns are shared characteristics that occur in each individual problem. What similarities do you observe? Finding these similarities in small decomposed problems can help us solve complex problems more efficiently.

4.6.3 Abstraction

"Abstraction" refers to focusing on important information only, ignoring irrelevant details. To reach a solution, we need to ignore unnecessary characteristics in order to focus on relevant details. It helps us identify specific similarities and differences between similar problems to work towards a solution.

So what is this important information that we need to focus on? In abstraction, the focus is mainly on general characteristics that are common to each element, instead of specific details.

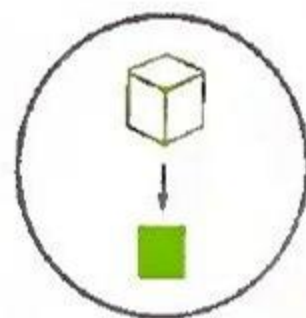


Fig. 4.10: Abstraction

Common examples of abstraction process are:

- A world map is an abstraction of the earth in terms of longitude and latitude, helping us describe the location and geography of a place.
- A sign of an aisle in a store—e.g., Walmart—is an abstraction of the items available in that aisle.
- When we write a book report, we summarize and discuss only the theme or key aspects of the book; it is abstraction.

Once we have the general characteristics, we can create a “model” of the problem; a model being the general idea of the problem we are trying to solve.

Once we have a model, we can design an algorithm.

4.6.4 Algorithm Writing

We have broken down the big problem into smaller, easily manageable problems. We have identified similarities between these problems. We have focused on relevant details and ignored anything irrelevant.

Now it is time to develop step-by-step instructions to solve each of the smaller problems, or the rules to follow when solving the problem.

These simple steps or rules are used to program a computer to help solve a complex problem in the best possible way. They are also called “algorithms”.

Definition: An algorithm is a plan, a set of step-by-step instructions used to solve a problem.

Writing an algorithm requires extensive planning for it to work correctly. The solution that the computer offers is as good as the algorithm we write. If the algorithm is not good, then our solution will not be good either.

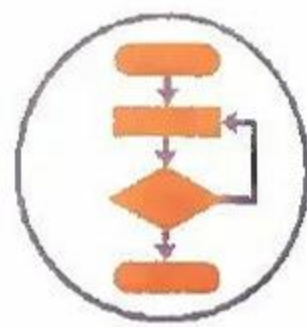


Fig. 4.11: Algorithm Writing

4.7 PYTHON PROGRAMMING LANGUAGE

When we encounter the word ‘Python’, it reminds us of the frightening reptile. However, the programming language has been coined ‘Python’ because just as the reptile has a flexible body, the language too is easy, simple and highly flexible.



Python is an open source, object-oriented, high-level programming language. It was developed by **Guido van Rossum** in 1991 at the National Research Institute for

Mathematics and Computer Science, the Netherlands. It is presently owned by Python Software Foundation (PSF).



Guido van Rossum

Python is based on the ABC language, a teaching language created to replace the programming language BASIC, which was developed earlier.

Python is a general-purpose programming language that can be used effectively to build any kind of program that does not require direct access to the computer hardware.

CTM: Python is based on the ABC language and the name was inspired by the famous BBC comedy show *Monty Python's Flying Circus*.

4.7.1 Features of Python

python is an interactive, interpreted language with simple syntax. It has become increasingly popular because of its salient features which are listed below:

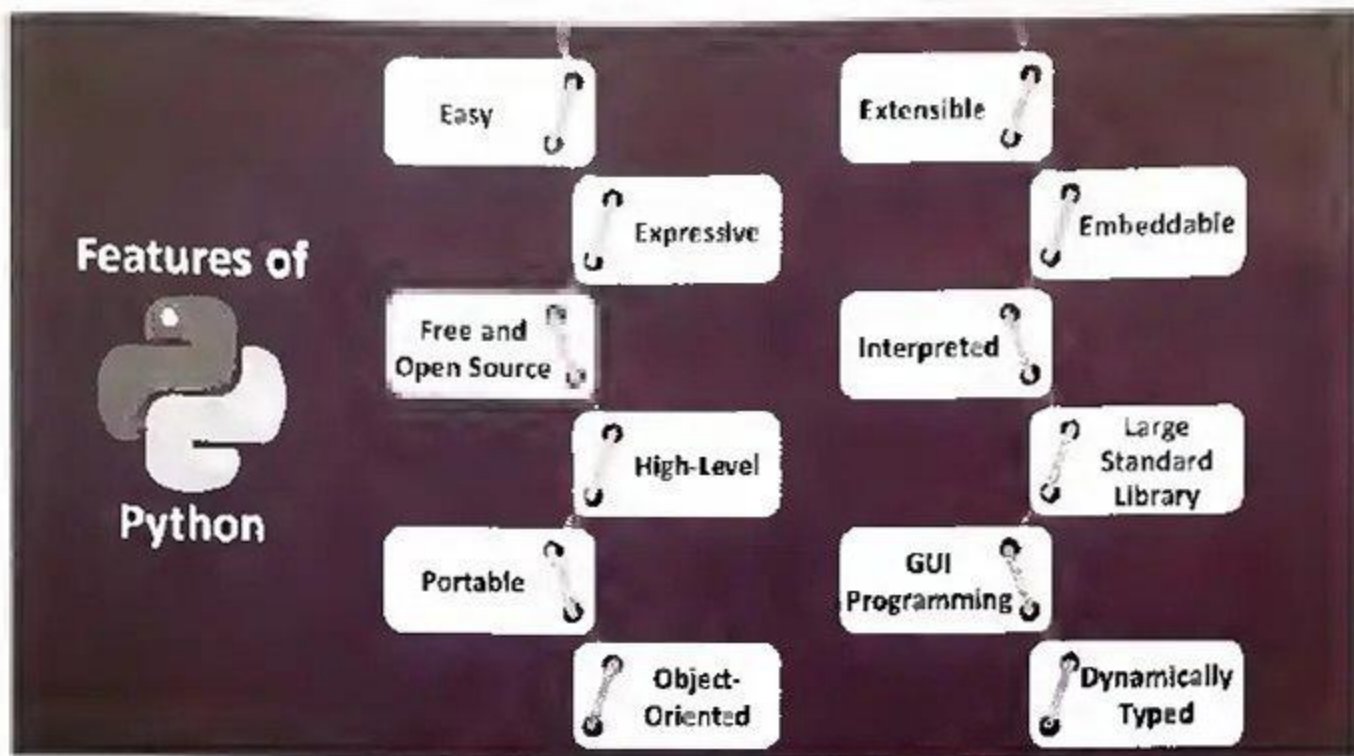


Fig. 4.12: Salient Features of Python

- Python is interpreted, interactive, directly executed, with pre-compiled code. This means that it is processed at run-time by the interpreter and we need not compile the program before executing it.
- It is a loosely typed Object-Oriented Programming Language with few keywords (reserved words) and simple English-like structure, and is easy to learn.
- It is a free open-source and portable language having a large repository of libraries.
- It takes less time to develop as Python programs are typically 3-5 times shorter than equivalent Java programs. This is because of its built-in high-level data types and its dynamic typing.
- It is extensible/extendable and highly efficient as there is no wastage of time in declaring the types of variables or arguments.
- It supports GUI (Graphical User Interface) and Garbage collection (better memory management).
- It is easily compatible with other languages like C, C++, Core Java, etc.
- Python is used for both scientific and non-scientific programming.

4.7.2 Advantages of Python

Apart from the above salient features, Python offers the following advantages:

- Platform-Independent:** It is platform-independent and can run across different platforms like Windows, Linux/Unix, MacOS and other operating systems.
- Readability:** Python programs use clear, simple, concise and English-like instructions that are easy to read and understand even by non-programmers or people with no substantial programming background.
- Object-Oriented Programming Language:** It is an interactive, interpreted and Object-Oriented Programming Language.

- (d) **Higher Productivity:** Since Python is a simple language with small codes and extensive libraries, it offers higher productivity to programmers than languages like C++ and Java. So, you write less and get more done.
- (e) **Less Learning Time:** Because of a simple and shorter code, lesser time is required to understand and learn Python programming.
- (f) **GUI Programming:** Python supports GUI applications that can be created and ported to many system calls, libraries and Windows systems such as Windows MFC (Microsoft Foundation Class Library), Macintosh and the X Window system of Unix.
- (g) **Ample Availability of Libraries:** It provides large standard libraries to solve a task.
- (h) **Syntax Highlighting:** It allows to distinguish between input, output and error messages by different colour codes.

4.7.3 Limitations of Python

Apart from several advantages that Python offers which make it one of the most popular programming languages, it has a few limitations as well.

- (a) **Speed:** Python is slower than C or C++. However, Python is a high-level language unlike C or C++; it's not closer to hardware.
- (b) **Mobile Development:** Python is not a very good language for mobile development. It is seen as a weak language for mobile computing. This is the reason very few mobile applications are built in it.
- (c) **Memory Consumption:** Python is not a good choice for memory-intensive tasks. Due to the flexibility of the data types, Python's memory consumption is also high.
- (d) **Database Access:** Python has limitations with database access. As compared to popular technologies like JDBC (Java Database Connectivity) and ODBC (Open Database Connectivity), Python's database access layer is found to be a bit underdeveloped and primitive. It cannot be applied in the enterprises that need smooth interaction of complex legacy data.
- (e) **Runtime Errors:** Python programmers have cited several issues with the design of the language. Because the language is dynamically typed, it requires more testing and has errors that only show up at run-time.

4.8 INSTALLING PYTHON

Ever since Python language was first developed, it has been becoming popular day by day among programmers. Several versions of Python have been released till now, beginning with Python 1.0.

We shall be working on Python 3.6.5; however, any 3.X.X version can be used to implement Python programming concepts as per CBSE guidelines. Before we start working, Python needs to be installed on our system. To install Python, we must first download the installation package of the required version from the link/URL given below:

<https://www.python.org/downloads/>

For installation details, scan



This Python distribution comes with Python IDLE, Python Interpreter and PIP (Python Package Manager).

4.9 INTERACTING WITH PYTHON (PYTHON IDLE)

Python is a flexible, dynamic, powerful language that can be used in different ways. It can be interactively used simply to test a code or a statement line by line or while exploring its interface.

Python is used by Google search engine, YouTube, Netflix, Spotify, Dropbox, Instagram, etc. It is being widely used for System programming, GUI programming, Internet Scripting/Web, Gaming, Text processing, Network programming, Commercial Robots and in Space and Scientific applications.

To write and run Python programs interactively, we can either use the command line window or the IDLE. IDLE is a simple Integrated Development Learning Environment that comes with Python. The most important feature of IDLE is that it is a program that allows the user to edit, run, browse and debug a Python program from a single interface.

4.9.1 Python Shell

When we first start Python IDLE by clicking on its icon created on the desktop or menu item on the **START** menu -> **Apps by name** -> **IDLE (Python 3.6.5 32-bit)** option, it always starts up in the shell.

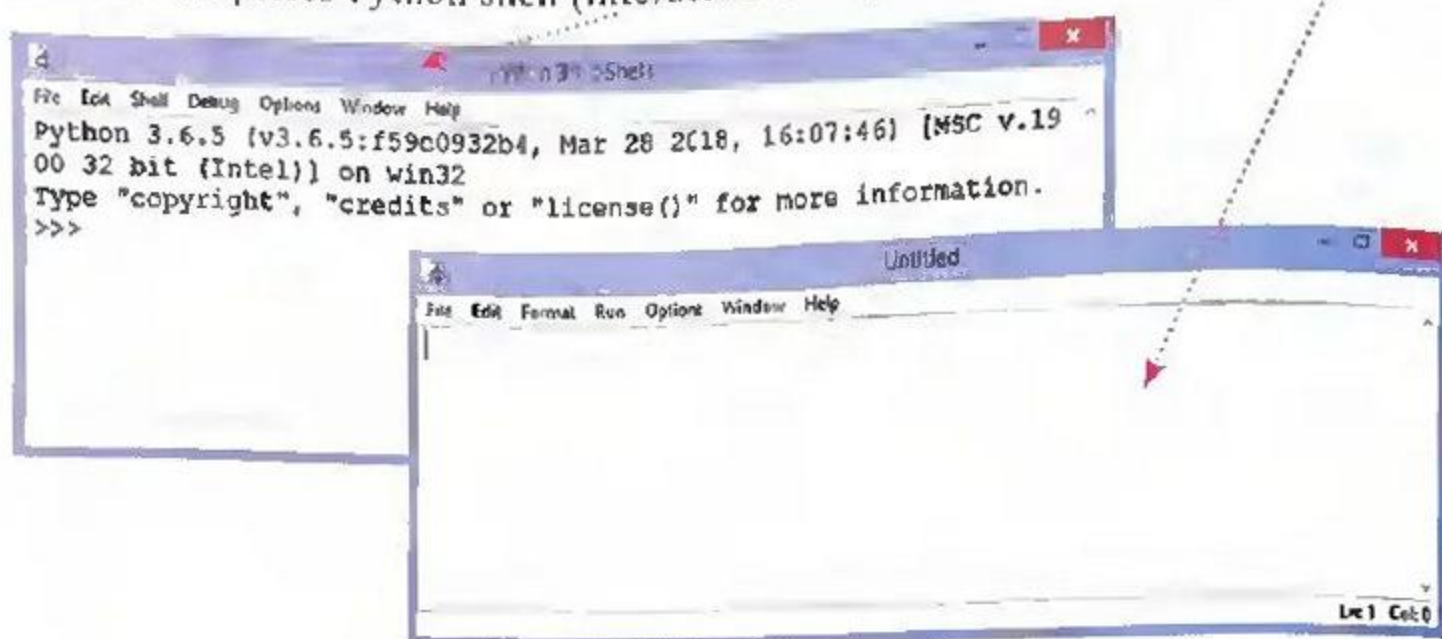


Fig. 4.13: Starting Python Shell

Python shell is an interactive window where we can type in the Python code and see the output in the same window. It is an interface between Python commands and the OS.

CTM: The interactive interpreter of Python is termed as Python Shell.

Python IDLE comprises Python shell (Interactive mode) and Python Editor (Script mode).



On opening, Python shell shows a welcome message displaying its version and a copyright notice. After this, the command prompt (>>>) followed by a blinking cursor gets displayed, which indicates that IDLE is now ready to take Python commands from the user (Fig. 4.14(a)).

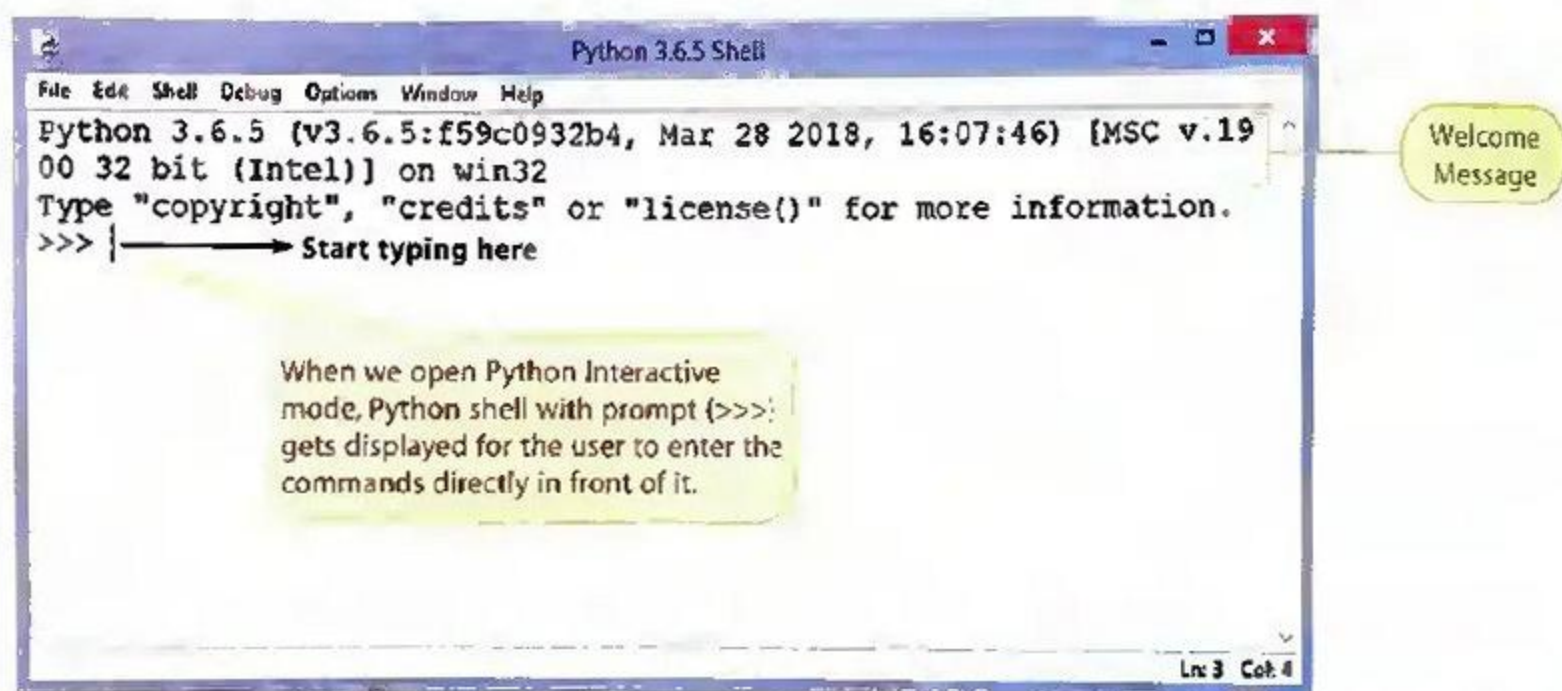


Fig. 4.14(a): Interactive Python Shell

This is the Python shell, which is part of Python's Integrated Development Environment. The three greater than signs (>>>) are called the prompt or Python command prompt.

Learning Tip: Python shell displays the prompt >>> to indicate that it is waiting for a user command to be entered.

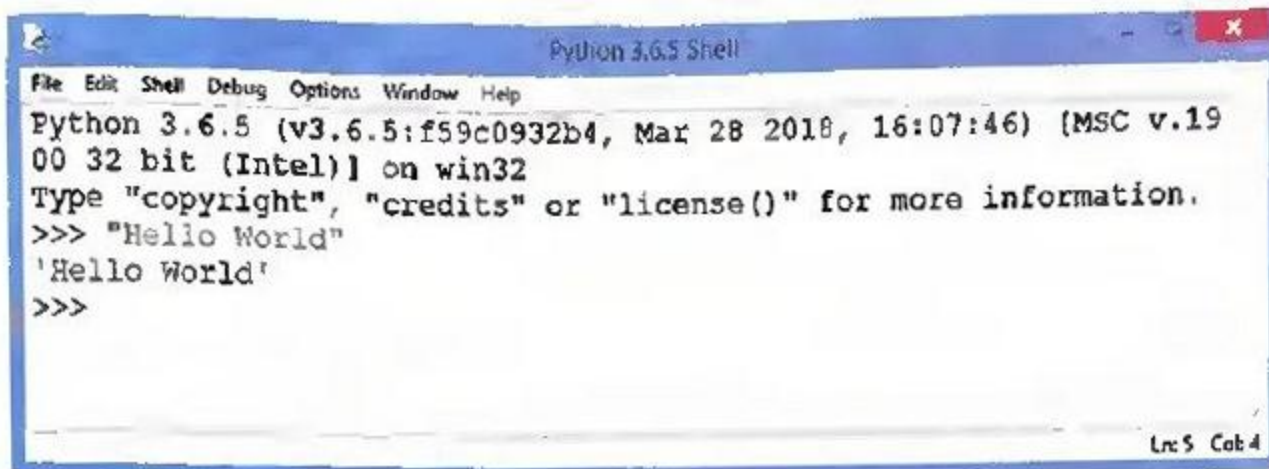
4.9.2 Command Line Interaction

When commands are entered directly in IDLE from the keyboard, the Interpreter/IDLE is said to be in Interactive mode. In Interactive mode, type the command "Hello World" in front of the prompt (>>>) and press Enter. See what happens.

➤ Our First Program

So, we begin with our first command in Python Interactive mode as illustrated below.

Practical Implementation-1

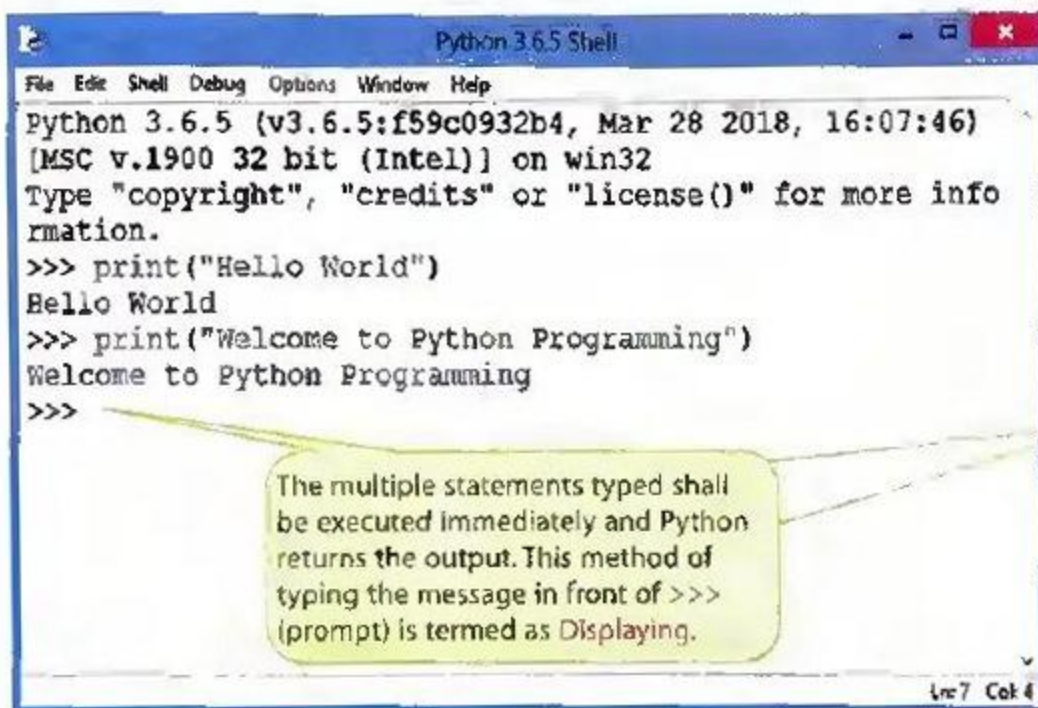


```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> "Hello World"
'Hello World'
>>>
```

Fig. 4.14(b): Displaying Message in Python Shell

As soon as we press the Enter key after typing our first statement, it gets executed immediately and the output is displayed then and there.

Alternatively, this can be done using the `print()` function as shown in Fig. 4.14(c) given below:



```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> print("Hello World")
Hello World
>>> print("Welcome to Python Programming")
Welcome to Python Programming
>>>
```

The multiple statements typed shall be executed immediately and Python returns the output. This method of typing the message in front of >>> (prompt) is termed as **Displaying**.

Example of multiple statements
>>> print(10)
>>> print(20)
10
20

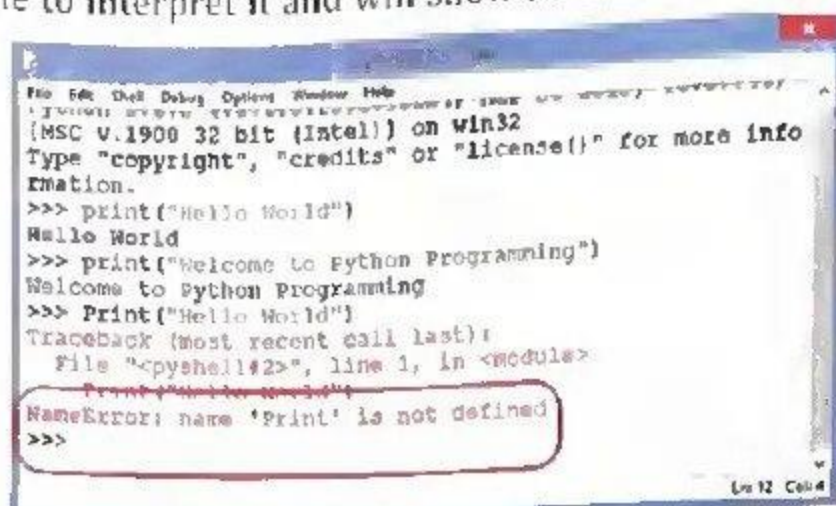
Fig. 4.14(c): Displaying Multiple Messages using `print()` function in Python Shell

In the above case, we have given two `print()` statements for displaying two different messages upon pressing the Enter key.

It has now become evident that we can give commands at the primary prompt and see immediate results. `print()` is a function which is used to display the specified content on the screen. The content (called argument) is specified within the parentheses. `print()` statement without any arguments will simply jump to the next line.

Python is a case-sensitive language. This means that Python differentiates between capital and small alphabets. For example, `Print` (P capital) and `print` (p small) are two different things for Python. Whereas `print` is a valid command in Python, `Print` is just a word and not a command. This can be observed by typing the command `Print("Hello World")`.

Python will not be able to interpret it and will show an error as shown in Fig. 4.14(d):



```
Python 3.6.5 Shell
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> print("Hello World")
Hello World
>>> print("Welcome to Python Programming")
Welcome to Python Programming
>>> Print("Hello World")
Traceback (most recent call last):
  File "<pyshell#2>", line 1, in <module>
    Print("Hello World")
NameError: name 'Print' is not defined
>>>
```

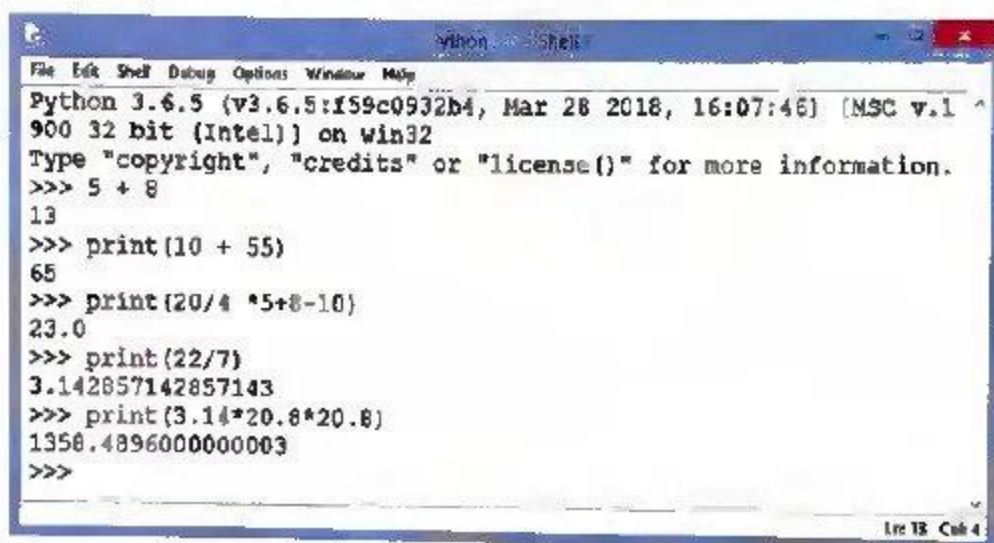
Fig. 4.14(d): Error on Typing Print() Instead of print()

We must remember that while using the `print()` command, the message or the argument to be displayed should be placed within quotation marks (single `'` or double `"`); the argument is then displayed as it is.

If the argument is not placed within quotation marks, then the interpreter tries to evaluate the argument and displays the result. This feature is used to perform calculations. If we give an arithmetic expression as an argument, the interpreter shows its value. This is shown in Fig. 4.14(e).

Practical Implementation-2

Using IDLE as a Calculator just like text/string messages we have given so far, we can also perform mathematical calculations using the Python command prompt.



```
Python 3.6.5 Shell
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> 5 + 8
13
>>> print(10 + 55)
65
>>> print(20/4 * 5 + 8 - 10)
23.0
>>> print(22/7)
3.142857142857143
>>> print(3.14 * 20.8 * 20.8)
1358.4896000000003
>>>
```

Fig. 4.14(e): Evaluating Arithmetic Expressions

In the above-entered statements, we have performed mathematical calculations using `print()` statement as well as by directly typing the command without the use of `print()` command. In either of the cases, these statements are immediately executed by the Python interpreter and the output is displayed within a fraction of a second.

CTM: To repeat a command in IDLE, press `Alt+p` in order to save time and typing effort.

Note: `print()` function will convert the argument or message into string before writing to the screen.

Consecutive arguments are separated by commas in `print()` function. When `print()` is executed, the values of consecutive arguments are separated by a space between them. So, we can say that space is the default separator of values in the output. We can also specify some other string as the separator using the `sep` argument of `print()` function (Fig. 4.14(f)).

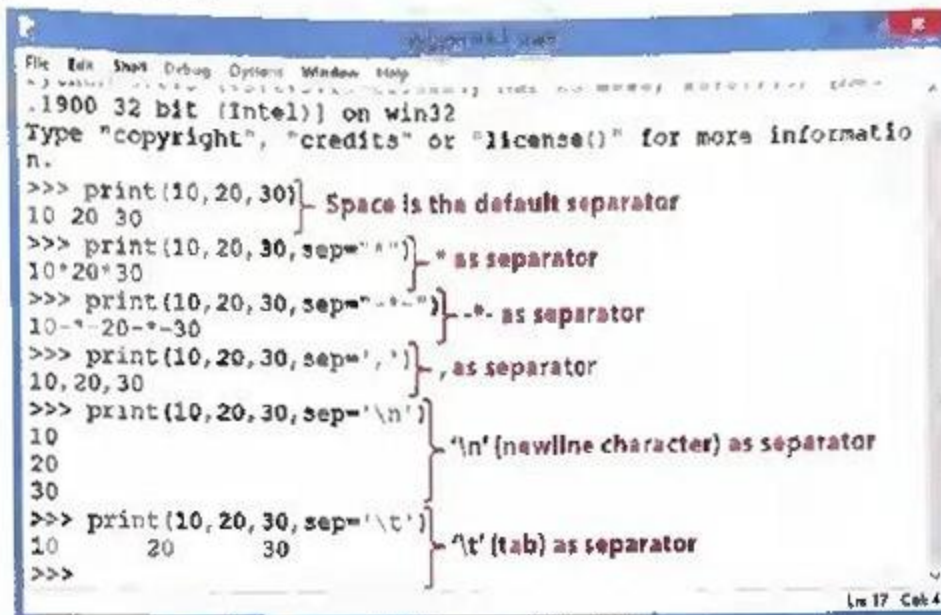
Syntax of print() function:

```
print(value, ..., sep=' ', end='\n', file=sys.stdout)
```

sep: String inserted between values, default, a space.

end: String appended after the last value, default, a new line

file: It is optional. By default, file attribute is used to print objects on the standard output device (sys.stdout), i.e., screen.



```
File Edit Shell Debug Options Window Help
Python 3.6.5 Shell
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:
86) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more informatio
n.
>>> print(10,20,30)
10 20 30
>>> print(10,20,30,sep='*')
10*20*30
>>> print(10,20,30,sep='--')
10--20--30
>>> print(10,20,30,sep=',')
10,20,30
>>> print(10,20,30,sep='\n')
10
20
30
>>> print(10,20,30,sep='\t')
10      20      30
>>>
```

Fig. 4.14(f): Use of 'sep' with print()

If we use the sep argument to specify the separator, then it must be specified after all the values to be printed. Otherwise, the interpreter shows a syntax error. An example is given below:

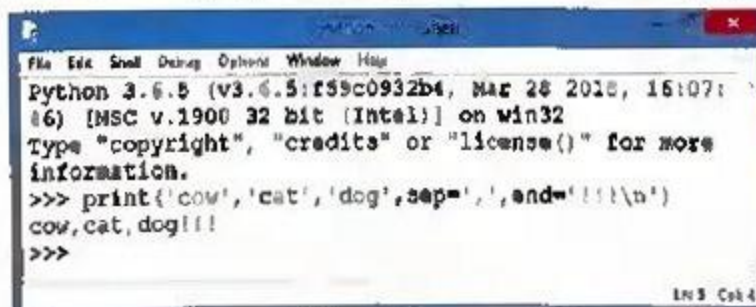
```
>>> print(10,20,30,sep = '*',40)
Syntax Error: positional argument follows keyword argument
```

End Parameter with print()

Python's print() function comes with a parameter called 'end'. By default, the value of this parameter is '\n', i.e., the new line character. We can end a print statement with any character/string using this parameter.

The print function uses sep to separate the arguments and end after the last argument. For example, type the following statement using "end" parameter with the print() function

>>> print('cow', 'cat', 'dog', sep=', ', end='!!!\n') and press the enter key. Python shell will execute it and display the output shown below (Fig. 4.14(g)):



```
File Edit Shell Debug Options Window Help
Python 3.6.5 Shell
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:
86) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more informatio
n.
>>> print('cow', 'cat', 'dog', sep=', ', end='!!!\n')
cow, cat, dog!!!
>>>
```

Fig. 4.14(g): Use of "end" Parameter with print()

More about Python Shell

The Python Shell Window (as shown in Fig. 4.14(a)) comprises drop-down menus and a Python command prompt (>>>) where you can type or enter statements or expressions to be executed. It is quite different in comparison to others in the way that it allows to scroll back to previous commands, to cut, copy and paste earlier commands/statements and to modify them.

The Python shell window has the following menu items: File, Edit, Shell, Debug, Options, Windows, and Help.

The **File menu** provides different options such as Open a new file, Open an existing file, Open Module, Recent Files, Save, Save As, Print, Close and Exit.

The **Edit menu** allows us to edit the text we type in the shell by applying various editing options such as Undo, Redo, Cut, Copy, Select All, Paste, Find and Replace.

The Shell and Debug menus are useful when creating larger programs. The **Shell menu** allows us to restart the shell or search the shell's log to trace the most recent reset. The **Debug menu** provides useful menu items for debugging the Python program in order to trace errors while checking for line-by-line execution of the entered Python statements.

The **Options menu** allows the user to configure IDLE according to the suitability and working requirements. For example, changing font style of the Python shell window, highlighting code colour, shortcut keys, etc.

The **Help option** provides Python help and documentation.

4.9.3 Python Editor Window (Working in Script Mode)

Till now, we were working with the Interactive mode where there is no provision for holding or saving the commands that we typed at the Python prompt. Moreover, the output is sandwiched between the command lines as shown in Fig. 4.14(f). All these commands, along with their output, get cleared as soon as we close the Shell window. This limitation can be overcome by working in the Python script mode where we can save the commands we are typing. This switchover is carried out through Python shell itself. Working with Python script mode involves the following steps:

Step 1: Creating Script/Program File

- The **File window/option** provides various options to create a new file, open an existing file, open a module, save the current module/session. When you click on the "New File" option (Fig. 4.15(a)), a new window opens, i.e., the Python script mode or Python Editor Window, which is a simple and standard text editor where you can type or edit your code. At the time of its opening, this file window is named 'Untitled' by default, as shown in Fig. 4.15(b). The name can be changed by the user by renaming the file.

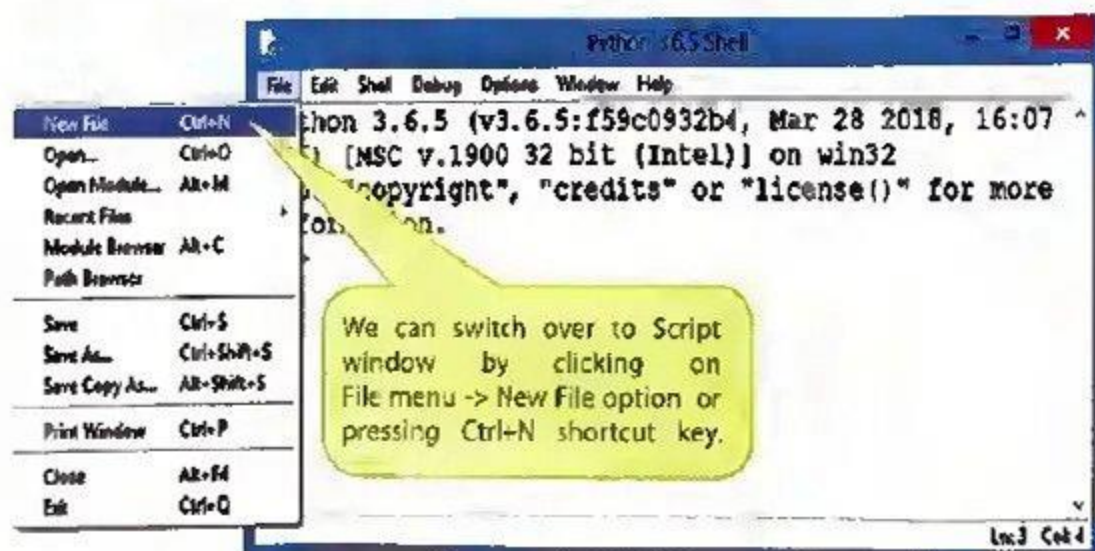


Fig. 4.15(a): Selecting File -> New File from Python Shell

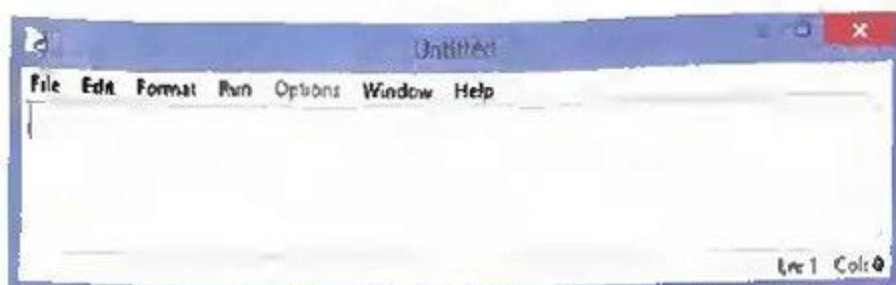


Fig. 4.15(b): Python Editor (Script Mode) Gets Opened

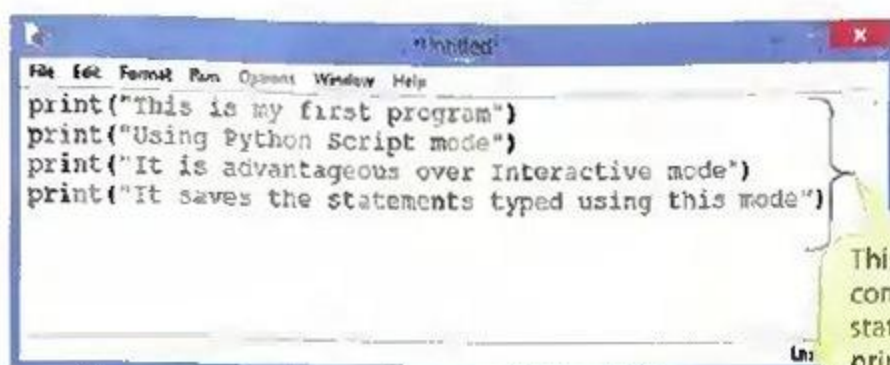


Fig. 4.15(c): Type the Commands in New Blank File (Script Mode)

Step 1(a): Our next step is to save this file by clicking on **File** menu -> **Save As** option, followed by typing a proper name for it in lieu of the default name "Untitled".

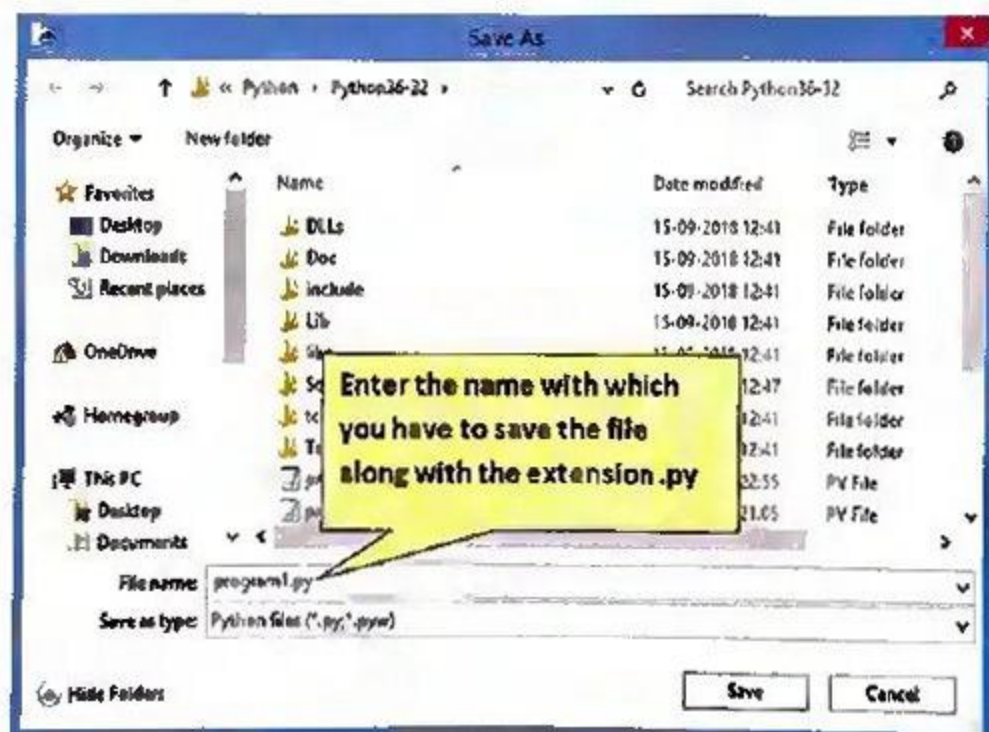
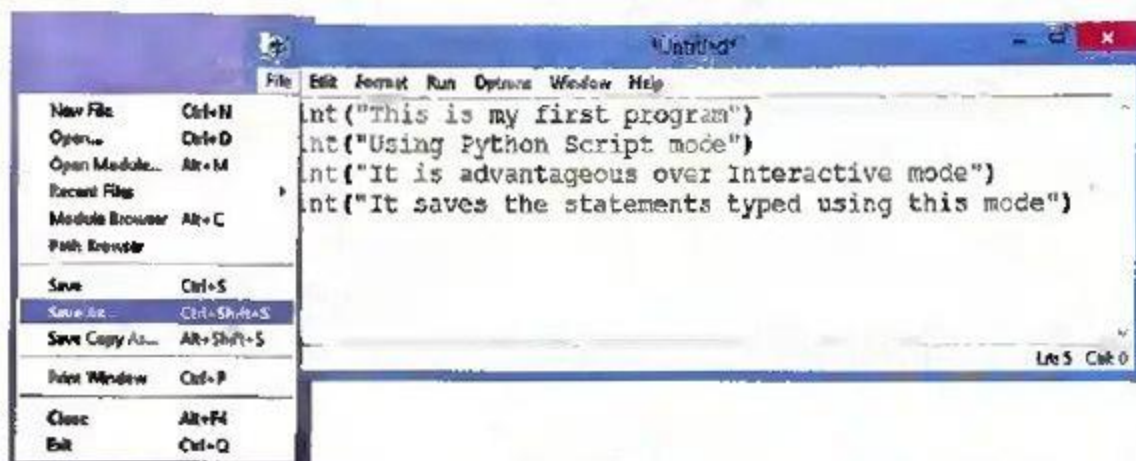


Fig. 4.15(d): Save your File with Extension .py

Even if we forget to give the extension .py or .pyw while saving the file for the first time, the Python Compiler automatically gives the extension along with the file name.

Step 2: Running Script/Program File

To execute a Python Script file, we have to choose the "Run Module" option from the Run menu or press the function key F5 (Fig. 4.15(e)).

More about Script Mode

While working in the script mode, when we enter the statements to be executed and press Enter key, we won't automatically see the results the way we would see in the Interactive mode. To see output from a script mode, we have to run this script and/or invoke the print() function.

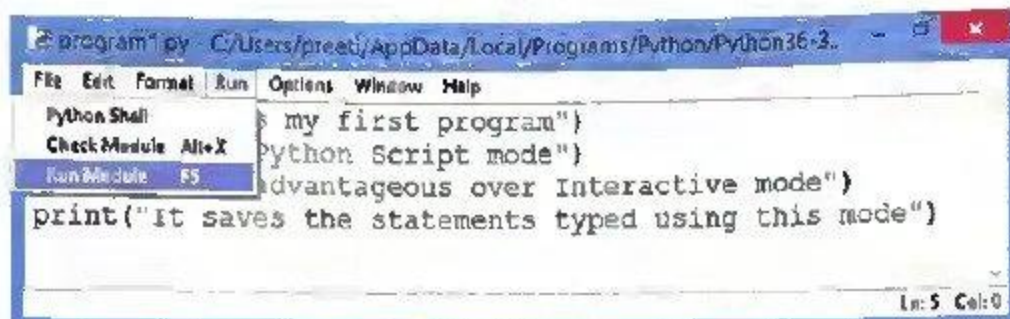


Fig. 4.15(e): Running the File

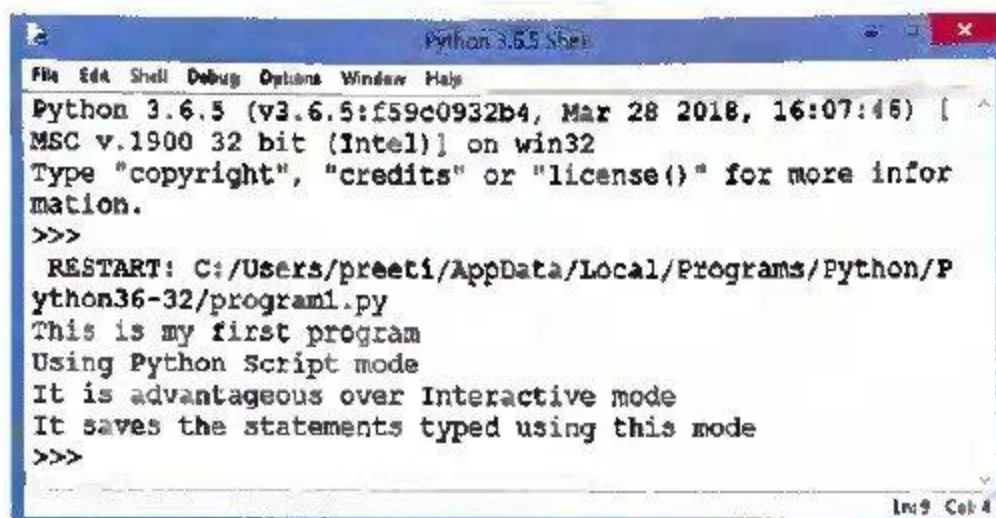


Fig. 4.15(f): Output of the Program Displayed Inside Python Shell

It must be remembered that even if we enter the print() statements in the script mode, the output shall be displayed only in Python shell. Also, all commands are given together in the form of a module/program/script and all output lines are displayed together which offers better understanding and clarification of the output obtained and, hence, no more cluttering between command and output takes place.

4.10 EXITING PYTHON

In order to exit the Python command prompt, Press Ctrl+q shortcut key or type quit() or exit() built-in function and press the Enter key.

POINT TO REMEMBER

If exit() or quit() does not work, then alternatively you can type the following commands:

```
import sys; sys.exit()
```



MEMORY BYTES

- Computational Thinking allows us to take a complex problem, understand what the problem is, and develop possible solutions.
- Decomposition, pattern matching, abstraction and algorithms are the four pillars of Computational Thinking.
- Decomposition is the process of breaking down a complex problem or system into smaller, more manageable parts.
- Pattern matching refers to looking for similarities among and within problems.
- Algorithm refers to the rules to follow to solve a problem.
- An algorithm is defined as a step-by-step procedure designed to perform an operation which will lead to the desired result, if followed correctly.
- Algorithms have a definite beginning and a definite end, and a finite number of steps.
- A good algorithm, which is precise, unique and finite, receives input and produces an output.
- Computational Thinking enables us to work out exactly what to tell the computer to do.
- A flow chart is a type of diagram that represents the algorithm graphically using boxes of various kinds, in an order connected by arrows.
- An algorithm where all the steps are executed one after the other, is said to execute in sequence.
- Decision-making involves selection of one of the alternatives based on outcome of a condition.
- There can be more than one approach to solve a problem and hence we can have more than one algorithm for a particular problem.
- The choice of algorithm should be made on the basis of time and space complexity.
- A complete set of instructions written using a programming language is termed as a Program/Code/Program Code.
- Python is a powerful and flexible programming language. It uses concise and easy-to-learn syntax which enables programmers to write more codes and develop more complex programs in a much shorter time.
- Python is a platform-independent programming language.
- Python interpreter executes one statement (command) at a time.
- Python provides two different ways to work with – Interactive mode and Script mode.
- Interactive mode does not save commands in the form of a program and the output is placed between commands as it is displayed as soon as we press the Enter key after typing one or more commands.
- Interactive mode is suitable for testing code.
- Python is a case-sensitive language. Thus, Ram and ram are two different names in Python.
- Python is an interpreted language.
- Python's Interactive Interpreter is also called Python Shell.
- The programs written in Python are easily readable and understandable.
- Wrong indentation can give rise to syntax error(s).
- Most Python editors will automatically indent the statements.
- print() statement outputs an entire (complete) line and then goes to the next line for subsequent output(s).
- Script mode is useful for creating programs and then running them later and getting complete output.
- All the programs in Python are saved with the .py or .pyw extension.

OBJECTIVE TYPE QUESTIONS

1. Fill in the blanks.

- (a) are a set of instructions given to the computer to solve a problem.
- (b) The set of techniques and approaches undertaken to solve problems logically is termed as
- (c) Python was created by
- (d) Python is called language.
- (e) Python syntax is
- (f) In Python, a syntax error is detected by the at
- (g) Python is independent language.
- (h) To start Python from the command prompt, use the command
- (i) Python programs are saved using the or extension.
- (j) Python programming can be done in and modes.
- (k) A problem that at first stroke is not easy to understand and solve is termed as a problem.
- (l) There are no standard rules to write
- (m) An is the process to write step-wise solution of a given problem in English-like language.
- (n) is the process of breaking down a complex problem into smaller problems so that they can be managed easily.

2. State whether the following statements are True or False.

- (a) Python is a dynamically typed language.
- (b) Alt+R is used to repeat the last typed command in Python shell.
- (c) Print and print are the same in Python.
- (d) sep argument is used with print() method.
- (e) Python is a compiled high-level language.
- (f) Python is a cross platform language.
- (g) Computational Thinking refers to the collective thought process involved in formulating problems and solutions.
- (h) Abstraction deals with focusing on background details of a problem.
- (i) Python Editor refers to Script mode in Python.
- (j) Ctrl+q shortcut key combination is used to exit Python.

3. Multiple Choice Questions (MCQs):

- (a) Python was developed by
 - (i) Charles Babbage
 - (ii) Guido van Rossum
 - (iii) Tim Berners Lee
 - (iv) Robert E. Kahn
- (b) What is Computational Thinking?
 - (i) Giving instructions to a computer
 - (ii) Working with binary language
 - (iii) Thinking like a computer
 - (iv) A set of techniques and approaches to solve problems
- (c) Python Language is
 - (i) Free
 - (ii) Open Source
 - (iii) Free & Open Source
 - (iv) Proprietary
- (d) Identify the correct print() statement:
 - (i) print(Hello)
 - (ii) print("Hello")
 - (iii) print('Hello')
 - (iv) print("Hello')
- (e) Which of the following is NOT a Computational Thinking technique?
 - (i) Coding
 - (ii) Decomposition
 - (iii) Pattern recognition
 - (iv) Abstraction

- (f) Which of the following is not an example of Computational Thinking?
- Taking all the requirements into consideration before starting a task.
 - Analyzing the bottlenecks involved.
 - Allowing any of the team members to take decisions for the entire team.
 - Taking into consideration how much time and money will be spent.
- (g) Python is a/an language.
- Compiled
 - Interpreted
 - Compiled & Interpreted
 - None of these
- (h) The interactive interpreter of Python is termed as
- Python shell
 - Python Script mode
 - Python Editor mode
 - Python command line
- (i) The three greater than signs (>>>) are called the Python
- Cursor
 - Command prompt
 - Pointer
 - Blinking cursor
- (j) Single line comments in Python begin with symbol.
- %
 - "
 - '
 - #
- (k) Which of the following codes is correct?
- ```
print("Programming is fun")
print("Python")
print("Computer Science")
```
  - ```
print("Programming is fun)
print("Python")
print("Computer Science)
```
 - ```
Print("Programming is fun")
print("Python")
print("Computer Science")
```
  - ```
Print("Programming is fun")
Print("Python")
Print("Computer Science")
```
- (l) Python is a case-sensitive language. This means
- Capital and small letters are same for Python.
 - Python doesn't care about the case of alphabets.
 - Python treats capital and small letters differently.
 - Python automatically capitalizes the small letters.

SOLVED QUESTIONS

1. What are the four main parts of Computational Thinking?

- Ans. • Decomposition
• Pattern Recognition
• Abstraction
• Algorithm Design

2. Can you spot a pattern in this set of numbers?

0, 1, 1, 2, 3, 5, 8, 13, 21, 34,...

Ans. Yes. This is a Fibonacci sequence. In programming terms, the sequence F_n of Fibonacci numbers, i.e., the third number, is the sum of two previous consecutive numbers and is defined by:

$$F_n = F_{n-1} + F_{n-2};$$

where $F_1 = 0$, $F_2 = 1$

with seed (initial or starting) values.

3. What is Python?

Ans. Python is a high-level, interpreted, dynamic, Object-Oriented Programming Language that supports GUI programming.

4. Why is Python interpreted?

Ans. Python is interpreted because the program is processed at run-time by the interpreter, i.e., converting source code into machine code line by line, and we do not need to compile the program before executing it.

5. Who developed Python?

Ans. Python was developed by Guido van Rossum in the early nineties at the National Research Institute for Mathematics and Computer Science in the Netherlands.

6. Write the full form of "IDLE".

Ans. Integrated Development Learning Environment

7. Why is Python easy to learn?

Ans. Python has relatively few keywords, simple structure and a clearly defined syntax. This allows anyone to understand and work easily in a relatively short period of time.

8. Write any one feature of Python library.

Ans. Python library is very portable and cross-platform compatible with UNIX, Windows and Macintosh.

9. Is Python a compiled or an interpreted language? [HOTS]

Ans. The normal execution of Python program is interpreted. However, subsets of the language can be compiled.

10. Python is a free and open-source language. What do you understand by this feature?

Ans. Python is a free and open-source language signifies that we do not have to pay anything to download it. Also, being an open-source language, its source code is also available if we wish to modify it.

11. State some salient features of Python.

Ans. Python is a modern, powerful, interpreted language with objects, modules, exceptions (or interrupts) and automatic memory management. It was introduced to the world by Guido van Rossum in the year 1991.

Salient Features of Python are:

- **Simple and Easy:** Python is a simple language that is easy to learn.
- **Free/Open Source:** Anybody can use Python without the need to purchase a licence.
- **High-level Language:** Being a high-level language, it can be easily understood by the user without the need to be concerned with low-level details.
- **Portable:** Python codes are machine and platform-independent.
- **Extensible and Embeddable:** Python programs support usage of C/C++ codes.
- **Standard Library:** Python standard library contains pre-written tools for programming.

12. Distinguish between Java and Python.

Ans. Java and Python can be distinguished on the following bases:

1. Python programs run slower than the Java codes, but Python saves much time and space. Python programs are 3 to 5 times smaller than Java programs.
2. Python is a loosely-typed dynamic language as no time gets wasted in declaring variable types as in Java.
3. Python is easier to learn than Java.

13. Describe flow charts and pseudo-code.

Ans. Flow charts are diagrams that show the step-by-step solution to a given problem.

Pseudo-code is a simple and informal way of writing programming code in English. There are no standard rules to write pseudo-code.

14. Write a pseudo-code to convert temperature in Celsius into Fahrenheit.

Ans. Input Celsius

Set Fahrenheit by multiplying Celsius by 1.8 and then adding 32

Print the Celsius and Fahrenheit.

15. Write a pseudo-code to calculate and display Area and Perimeter of a rectangle.

Ans. Input length of a Rectangle.

Input breadth of a Rectangle.

Set the area by multiplying length and breadth

Set the Perimeter by adding length and breadth and then

multiply it by 2. Display area and perimeter.

16. Write a pseudo-code to find the largest number among the three inputted numbers

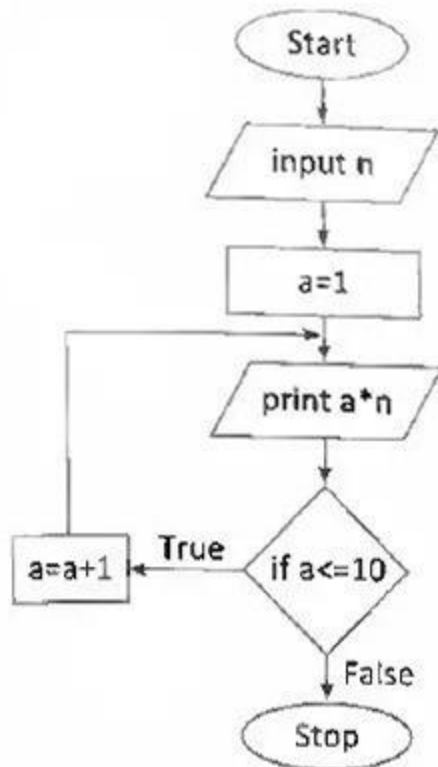
Ans. Input 3 numbers A,B,C.

If A is greater than B and A is greater than C,
then display 'A' is the largest number,
otherwise display 'C' is the largest number.

Otherwise if B is greater than C, then display 'B' is the largest number,
otherwise display 'C' is the largest number.

17. Draw a flow chart to accept a number and display its table.

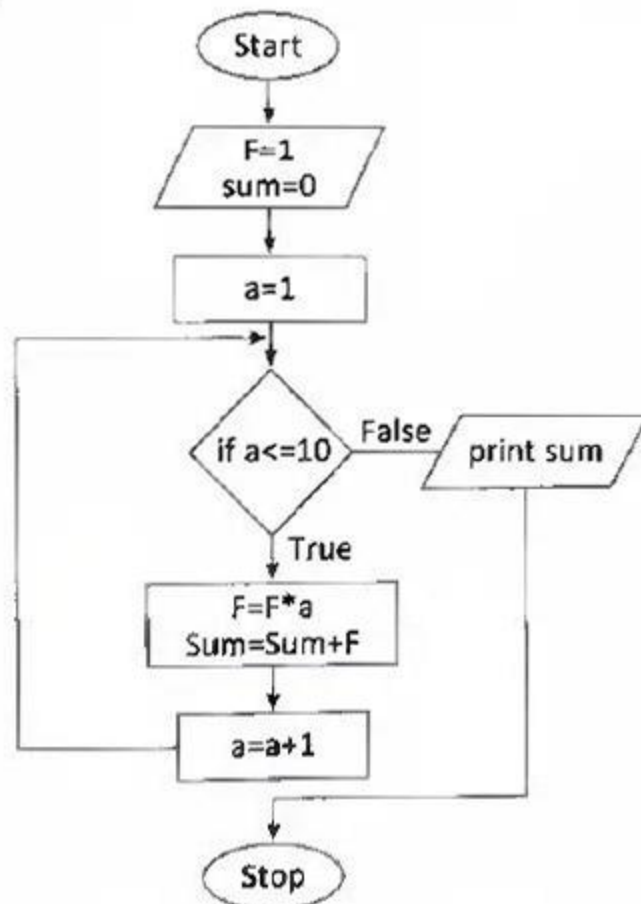
Ans.



18. Draw a flow chart to display sum of the following series:

$S=1!+2!+3!+...+10!$

Ans.



19. What is the difference between Interactive mode and Script mode in Python?

Ans. In Interactive mode, instructions are given in front of Python prompt (`>>>`) in Python Shell. Python carries out the given instructions and shows the result there itself.

In Script mode, Python instructions are stored in a file, generally with .py extension, and executed together in one go as a unit. The saved instructions are known as Python script or Python program.

20. Give the output for the following:

(i) `>>>print('apple','mango','orange','banana',sep='@')`

(ii) `>>>print(1,2,3,4,5,end='*****',sep='%')`

(iii) `>>>print(2,4,6,8,10);print(3,6,9,12,15)`

Ans. (i) apple@mango@orange@banana

(ii) 1%2%3%4%5*****

(iii) 2 4 6 8 10

3 6 9 12 15

21. Give the output for the following:

1. `print("Hello")`

2. `print(5.5)`

3. `print(4+6)`

Ans. `>>> print("Hello")`

Hello

`>>> print(5.5)`

5.5

`>>> print(4+6)`

10

22. Can you come up with an algorithm to make a cup of tea?

Ans. (i) Put water into kettle

(ii) Turn kettle on

(iii) Get a cup

(iv) Put tea bag into cup

(v) When water gets boiled, add to cup

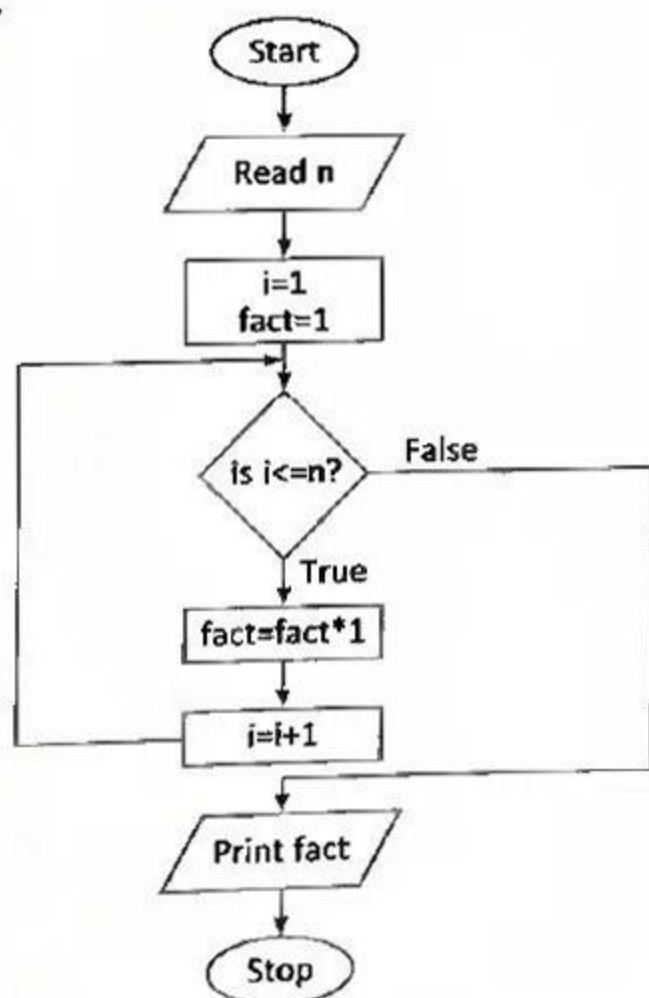
(vi) Add milk and sugar if needed

(vii) Stir with spoon

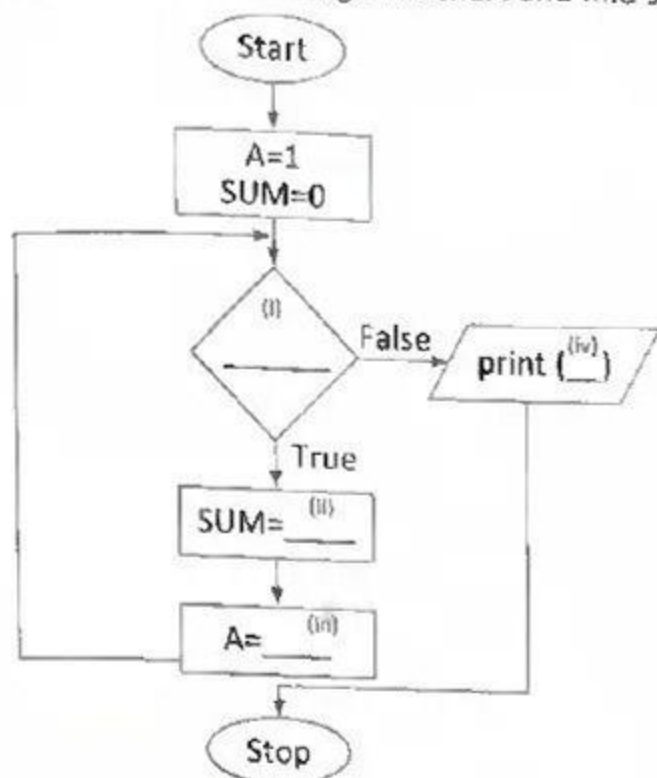
(viii) Remove tea bag and drink the tea

23. Draw a flow chart to accept a number and find its factorial.

Ans.



24. Complete the following flow chart and find sum of first 20 natural numbers.



- Ans. (i) if $A \leq 20$
 (ii) $SUM = SUM + A$
 (iii) $A = A + 1$
 (iv) print SUM

25. Define the term Computational Thinking.

Ans. Computational Thinking allows us to take a complex problem, understand what it is, and develop possible solutions. We can then present these solutions in a way that a computer, a human, or both, can understand.

26. What is Decomposition?

Ans. Decomposition is one of the four cornerstones of Computational Thinking. It involves breaking down a complex problem or system into smaller parts that are more manageable and easy to understand. The smaller parts can then be examined and solved, or designed individually, as they are simpler to work with.

27. Why is Decomposition important?

Ans. If a problem is not decomposed, it is much harder to solve. Dealing with many different stages all at once is much more difficult than breaking down a problem into a number of smaller problems and solving each problem one at a time. Breaking down the problem into smaller parts means that each smaller problem can be examined in more detail.

28. Why do we need Pattern Recognition?

Ans. Pattern Recognition is one of the four cornerstones of Computational Thinking. It involves finding the similarities or patterns among small, decomposed problems that can help us solve more complex problems efficiently.

29. Explain the characteristics of Computational Thinking.

Ans. The salient features of Computational Thinking are:

- (i) Formulating problems in a way that enables us to use a computer and other tools to help solve them.
- (ii) Logically organizing and analyzing data.
- (iii) Representing data through abstractions such as models and simulations.
- (iv) Automating solutions through algorithmic thinking (a series of ordered steps).
- (v) Identifying, analyzing and implementing possible solutions with the goal of achieving the most efficient and effective combination of steps and resources.
- (vi) Generalizing and transferring this problem-solving process to a wide variety of problems.

30. Why is abstraction important?

Ans. Abstraction allows us to create a general idea of what the problem is and how to solve it. The process instructs us to remove all specific details and any patterns that will not help us solve our problem. This helps us form our idea of the problem. This idea is known as a 'model'.

31. Is Python case-sensitive? What is meant by the term 'Case-sensitive' in programming languages?

Ans. Yes, it is. Text or typed input that is sensitive to capitalization of letters is called case-sensitive, for example, "Computer" and "computer" are two different words because the "C" is in the upper case.

UNSOLVED QUESTIONS

1. What are Computational Thinking skills?
2. How are Computational Thinking skills applied in finding solutions that can be interpreted into software applications?
3. Python is free and open source. What do you understand by this feature?
4. Write some limitations of Python.
5. What is the difference between script mode and interactive mode in Python?
6. Give an example of Computational Thinking.
7. How can one use Computational Thinking for problem-solving?
8. Differentiate between flow chart and algorithm.
9. Write pseudo-code that reads two numbers and divide one by another and display the quotient.
10. Two friends decide who gets the last slice of a cake by flipping a coin five times. The first person to win three flips wins the cake. An input of 1 means player 1 wins a flip, and a 2 means player 2 wins a flip. Design an algorithm to determine who takes the cake?
11. Write a pseudo-code to print all multiples of 5 between 10 and 25 (including both 10 and 25).
12. Write pseudo-code that will perform the following:
 - (a) Read the marks of three subjects: Computer Science, Mathematics and Physics out of 100
 - (b) Calculate the aggregate marks
 - (c) Calculate the percentage of marks
13. Write an algorithm to find the greatest among two different numbers entered by the user.
14. Write an algorithm that performs the following:

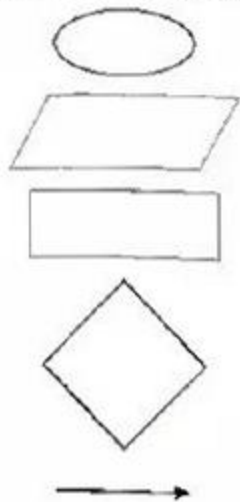
Ask a user to enter a number. If the number is between 5 and 15, write the word GREEN. If the number is between 15 and 25, write the word BLUE.

If the number is between 25 and 35, write the word ORANGE.

If it is any other number, write that ALL COLOURS ARE BEAUTIFUL.
15. Write an algorithm that accepts four numbers as input and find the largest and smallest of them.
16. What is the difference between Computational Thinking and Algorithmic Thinking?
17. Explain the components of Computational Thinking.
18. What is pattern matching?
19. "Decomposition leads to simplicity." How?
20. Explain the significance of planning in Computational Thinking.
21. Write an algorithm to find a factorial of an inputted number.

22. Match the pairs:

Flow chart Symbol



Functions

Flow of Control

Process Step

Start/Stop of the Process

Data

Decision-making

23. Write the pseudo-code to print the bill depending upon the price and quantity of an item. Also print Bill GST, which is the bill after adding 5% tax in the total bill.
24. When was Python released?
25. Who developed Python and which two languages contributed to Python as a programming language?
26. Is Python case-sensitive?
27. What is IDLE?
28. Differentiate between displaying and printing method in Python.
29. Briefly explain the salient features of Python.
30. What do you understand by cross-platform software with respect to Python?
31. What are the advantages of Python programming language?
32. In how many different ways can you work in Python?
33. What are the advantages/disadvantages of working in Interactive mode in Python?
34. Write instructions in Interactive mode for the following:
- (a) To display sum of 3, 8.0, $6*12$
 - (b) To print sum of 16, 5.0, 44.0
35. Write instructions to get the following result:
- Python is easy to learn and write.
It allows us to work in two modes: Interactive mode and Script mode.
Interactive mode is also known as Python Shell and Script mode is known as Python Editor. It is a platform-independent language.
We find it interesting to work with Python.
Do it in both Interactive mode and Script mode.
36. Write a code that prints your full name and your birthday as separate strings.
37. Record what happens when the following statements are executed:
- (a) `print(n=17)`
 - (b) `print(8+9)`
 - (c) `print(4.2, "hello", 6-2, "world", 15/2.0)`
38. Use IDLE to calculate:
- (a) $6+4*10$
 - (b) $(6+4)*10$

39. Try the following code on the Python shell and evaluate the output generated:

Python is easy to learn and write.

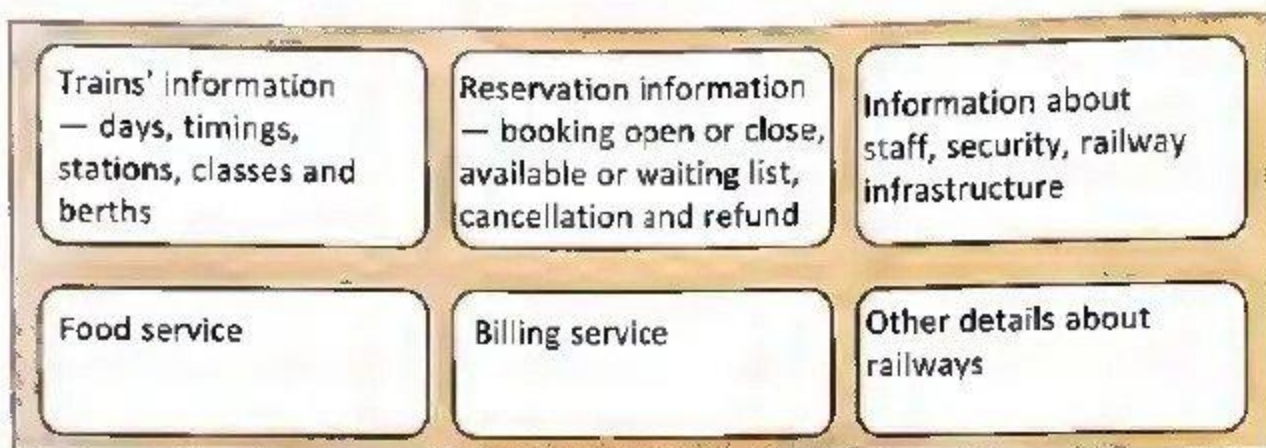
```
>>> print(3.14159*7)
>>> print('I am a class XI' + 'student')
>>> print('I', 'm')
>>> print("class XI student")
>>> print("I'm ", 16, "years old")
```

CASE-BASED/SOURCE-BASED INTEGRATED QUESTION

Railway Booking and Lodging is a travel agency that books railway tickets for its customers. They ask the user to furnish all the details about the date of journey, destination, preferred time of journey and other required relevant details. They need to computerize all their processing and working methodology.

You being a software analyst, develop a solution by decomposing the entire problem and design a suitable railway ticket reservation system using concept of decomposition.

- Ans. The entire system can be described and decomposed into various functionalities and subsystems as given below:



Railway Reservation System